# Mini Project Report
# on
# SMNS - Smart Multi-Note System

***Submitted by***

**Meenakshi M Kumar (20222066)**
**Swsthik H Nair (20222105)**
**Swarag V S (20222119)**
**Narthana Baby B S(20222076)**

***In partial fulfilment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering.***

**DIVISION OF COMPUTER SCIENCE AND ENGINEERING**
**SCHOOL OF ENGINEERING**
**COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY**

**APRIL 2025**

DIVISION OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

# CERTIFICATE

Certified that this is a Mini Project Report titled

**SMNS - Smart Multi-Note System**

Submitted by

Meenakshi M Kumar (20222066)
Swsthik H Nair (20222105)
Swarag V S (20222119)
Narthana Baby B S (20222076)

of VI Semester, Computer Science and Engineering in the year 2024 in partial fulfillment requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering of Cochin University of Science and Technology.

| | | |
|---|---|---|
| Mr.Pramod Pavithran | Mr. Pramod Pavithran | Ms.Sheena S |
| **Head of Division** | **Project Coordinator** | **Project Guide** |

# ACKNOWLEDGEMENT

First and foremost, we offer our deepest gratitude and humble prayers to the Almighty, whose grace and blessings have been our guiding light throughout this project. We express our heartfelt appreciation to our esteemed guide, Ms. Sheena S, for her invaluable guidance, unwavering patience, and continued support. Her insightful feedback and encouragement were instrumental in shaping the successful implementation of this project. We also extend our sincere thanks to Mr. Pramod Pavithran, Head of the Department and Project Coordinator, for his constant support and constructive guidance during every phase of our work. We remain truly grateful to our friends and classmates for their helpful suggestions, cooperation, and motivation throughout the development process. Their involvement played a vital role in refining our ideas and execution. Last but not least, we would like to express our profound gratitude to our families for their endless encouragement, understanding, and moral support. Their presence behind us made this accomplishment possible.

Meenakshi M Kumar (20222066)

Swsthik H Nair (20222105)

Swarag V S (20222119)

Narthana Baby B S (20222076)

# DECLARATION

We, Meenakshi M Kumar, Swsthik H Nair, Swarag V S, and Narthana Baby B S, hereby declare that the project titled **"SMNS – Smart Multi-Note System"** is a bona fide work carried out by us during the year 2025, under the guidance of **Mrs. Sheena S**, Assistant Professor, School of Engineering, CUSAT. We further declare that this project has not been submitted, either in part or full, for the award of any degree, diploma, associateship, fellowship, or any other academic recognition in any university or institution.

# ABSTRACT

The Smart Multi-Note System (SMNS) is a cross-platform academic solution designed to optimize modern digital learning by addressing limitations in traditional note taking. Integrates intelligent features to support students in capturing, understanding and organizing content efficiently. Key capabilities include real-time handwriting-to-text conversion, custom font rendering to retain handwriting style, and automated PDF content extraction to identify frequently asked questions. An AI-powered Q&A module enables contextual responses based on uploaded documents, while a voice recording analyzer transcribes lectures and summarizes spoken content. Furthermore, SMNS facilitates automatic quiz generation, summarization. Built on advanced Natural Language Processing (NLP) and Large Language Models (LLM), such as LLaMA 2, the system ensures accurate comprehension and content generation. In general, SMNS improves educational outcomes by promoting personalized learning, improving retention, and streamlining academic workflows.

# List of Figures


4.1 System Architecture of SMNS . . . . . 15
4.2 Dataflow Diagram Of SMNS . . . . . 17
4.3 Use Case Diagram of the System . . . . . 18
4.4 Data Upload Block . . . . . 21
4.5 Backend Python Snippet . . . . . 22
4.6 API custom prompting . . . . . 23

5.1 Home page of Web App . . . . . 26
5.2 Home page of Android App . . . . . 27
5.3 Hand Canvas . . . . . 28
5.4 Modes Of Data to Be Uploaded . . . . . 29
5.5 Chatbot . . . . . 30
5.6 Voice . . . . . 30
5.7 Quiz . . . . . 31
5.8 Ways to Share Notes . . . . . 32
5.9 App Routing script . . . . . 33
5.10 Voice Processing . . . . . 34
5.11 Pdf Upload . . . . . 35
5.12 Pdf Extract . . . . . 36

6.1 Checklisting . . . . . 39
6.2 Categorizing Notes . . . . . 40
6.3 Navigation . . . . . 41
6.4 Selecting Models . . . . . 43
6.5 Privacy . . . . . 47

# List of Tables

4.1 User Schema: Properties, Descriptions, and Data Types . . . . 19
4.2 Note Schema: Property Name, Descriptions, and Data Types . 19

7.1 Unit Testing Results for SMNS Modules . . . . . 49
7.2 Performance Testing Results . . . . . 51
7.3 Bug Fixes and Enhancements . . . . . 52

# Contents

**Acknowledgement** **i**

**Declaration** **ii**

**Abstract** **iii**

**List of Figures** **iv**

**List of Tables** **v**

**1 Introduction** **1**
1.1 Methodology . . . . . 1
1.2 Features . . . . . 3
1.3 Significance of Work . . . . . 3

**2 Literature Review** **5**
2.1 Handwriting Recognition in Digital Note Taking . . . . . 5
2.2 Whisper: Robust Speech Recognition System . . . . . 6
2.3 AI in Education: Personalized Learning Assistants . . . . . 6
2.4 Extractive and Abstractive Summarization Techniques . . . . 7
2.5 Multimodal Intelligence via Gemini API . . . . . 7

**3 System Study** **9**
3.1 Existing System . . . . . 9
3.2 Proposed System . . . . . 9
3.3 Software Requirements Specification . . . . . 10
3.3.1 Overall Description . . . . . 10
3.3.2 Product Functions . . . . . 11
3.4 Software Requirements Specification . . . . . 12
3.4.1 External Interfaces . . . . . 12

3.4.2 Functional Requirements . . . . . 13
3.4.3 Non-Functional Requirements . . . . . 14

**4 System Design** **15**
4.1 Data Flow Diagram (DFD) . . . . . 16
4.2 Use Case Diagram . . . . . 17
4.3 Database Design . . . . . 18
4.4 Model and Dataset . . . . . 19
4.5 Input-Output Design . . . . . 20
4.5.1 User Registration . . . . . 20
4.5.2 Voice Upload . . . . . 20
4.5.3 PDF Upload . . . . . 21
4.5.4 Handwriting Input . . . . . 21
4.5.5 AI Summary Generator . . . . . 22
4.5.6 AI Q&A Interface . . . . . 22
4.6 Modular Design . . . . . 23

**5 System Implementation** **25**
5.1 Modular Implementation . . . . . 26
5.1.1 Frontend Development (React JS + TypeScript) . . . 26
5.1.2 Backend Development (Flask) . . . . . 32
5.1.3 Database Integration (MongoDB) . . . . . 33
5.1.4 AI Model Integration . . . . . 33
5.1.5 Handwriting Input Module . . . . . 35
5.1.6 PDF Upload and Question Extraction . . . . . 35
5.1.7 AI-Driven Features . . . . . 36
5.1.8 Data Storage and Retrieval . . . . . 37

**6 Technologies, Stacks and Tools Used** **38**
6.1 Development Environment . . . . . 38
6.2 Frontend Technologies . . . . . 38
6.2.1 Mobile Application Frontend . . . . . 39
6.2.2 Web Application Frontend . . . . . 41
6.3 Backend and Data Management . . . . . 42
6.3.1 Mobile Application Backend . . . . . 42
6.3.2 Web Application Backend . . . . . 42
6.4 AI Integration Features . . . . . 43

6.5 Libraries and Dependencies . . . . . 44
6.5.1 Mobile Application Libraries . . . . . 44
6.5.2 Web Application Libraries . . . . . 45
6.6 Testing Tools . . . . . 45
6.7 Version Control and Project Management . . . . . 46
6.8 Deployment . . . . . 46
6.9 Security Measures . . . . . 46

**7 Testing** **48**
7.1 Unit Testing . . . . . 48
7.2 Integration Testing . . . . . 49
7.3 Performance Testing . . . . . 50
7.4 Usability Testing . . . . . 51
7.5 Bug Fixes and Improvements . . . . . 51

**8 Future Scope** **53**

**9 Conclusion** **54**

**References** **55**

# Chapter 1

# Introduction

In today's academic landscape, students are constantly challenged to keep up with fast-paced lectures, complex study materials, and fragmented note-taking methods. Traditional handwritten notes, though familiar, are often unstructured, difficult to share, and nearly impossible to search or analyze. On the other hand, digital alternatives often lack the intelligence and personalization that learners need. To address these limitations, we propose the Smart Multi-Note System (SMNS), an AI-powered mobile and web application designed to enhance how students interact with their notes. This app brings together technologies such as Natural Language Processing (NLP), machine learning, speech-to-text, and handwriting recognition to create a powerful note management system tailored for educational needs. Through real-time handwriting to customized font conversion, AI-driven Q & A, PDF analysis, and smart content generation, this app serves as an intelligent academic companion for students. The system also helps in extracting repeated questions, summarizing content, and even generating quizzes thereby bridging the gap between passive note storage and active learning.

## 1.1 Methodology

The development of the SMNS (Smart Multi-Note System) followed a structured methodology, beginning with a thorough requirement analysis. To understand the needs of the end-users students we conducted surveys and interviews to identify the primary challenges in note-taking and studying. This revealed recurring pain points such as disorganized notes, difficulty in revision, and the lack of tools to support visual learning preferences.

Based on these insights, we moved to the technology stack selection. The system's frontend was developed using React JS with TypeScript for enhanced maintainability and type safety. The Flask framework was chosen for backend development due to its simplicity and flexibility. For AI-powered features like summarization and question-answering, we integrated the Gemini API a large language model by Google. All user data, notes, and AI-generated content were stored securely in MongoDB, ensuring scalability and fast data access.

In the system architecture design phase, we adopted a modular approach, enabling the separation of concerns and ease of maintenance. Distinct components were designed to handle handwriting input, voice processing, and PDF document parsing. Each of these modules feeds into independent pipelines responsible for note conversion, quiz generation, and intelligent Q&A services. The project also involved a dedicated model training and fine-tuning stage, where we curated datasets of handwritten notes and lecture transcripts. These were used to fine-tune an open-source LLM to improve the quality of AI outputs tailored for educational content. And, we compiled datasets of repeated questions to support topic mapping and focused revision strategies.

The development and integration process was iterative and collaborative. The frontend interface was built to support various input types handwriting, audio, and PDFs while providing seamless communication with the backend through RESTful APIs. The AI models were tightly integrated to enable real-time processing and delivery of intelligent content such as summaries, quizzes, and contextual answers. Once development was complete, we entered the testing and evaluation phase. This included unit testing of individual modules, integration testing for system-wide workflows, and performance testing to validate responsiveness and scalability. Feedback from early-stage testers helped us refine both the functionality and usability of the application.

Finally, the system was prepared for deployment and maintenance. The application was deployed on cloud platforms to ensure high availability and adaptability to user demand. A feedback loop was established based on user analytics and system logs to facilitate continuous improvement and evolution of the system in response to user needs.

## 1.2 Features

The SMNS - Smart Multi-Note System is a smart, cross-platform solution designed to transform the academic experience by combining the power of artificial intelligence with intuitive user interfaces. It supports both web and mobile platforms, ensuring students can access and manage their notes anytime, anywhere. The app features real-time handwriting-to-text conversion with custom font rendering that mimics the user's original writing style, creating a personal touch in the digital environment. Students can upload PDF documents for analysis, where the system intelligently extracts repeated questions, helping them identify high-yield exam content.

The app also includes a voice recording analyzer that transcribes and summarizes lectures using the Whisper model, making it easier to review spoken content in text form. Powered by the Gemini API, the app introduces AI-enhanced tools that promote active and personalized learning. These include a context-aware Q&A assistant, capable of responding to user questions based on their uploaded notes, as well as the automatic generation of summaries, quizzes (MCQs and fill-in-the-blanks).

The seamless integration of these features helps students streamline their revision, improve comprehension, and retain information more effectively. Whether accessed through a mobile device or a desktop browser, the SMNS app ensures a consistent and powerful academic experience tailored for modern learners.

## 1.3 Significance of Work

The SMNS - Smart Multi-Note System addresses a critical need in modern education: transforming passive note-taking into active, personalized learning. In an era of information overload, students often struggle with organizing notes, identifying key concepts, and revising efficiently especially when materials are dispersed across handwritten notes, PDFs, and lecture recordings. This project leverages Google's Gemini API and React JS with TypeScript

to deliver an intelligent platform that simplifies content ingestion, enhances comprehension, and fosters deep learning. By integrating real-time handwriting conversion, voice transcription, summarization, question generation, the app bridges multiple learning styles auditory, visual, and kinesthetic.

The SMNS system provides a unified AI-powered solution for these issues by incorporating:

1) Summarization: Reduces note length by approximately 60% while retaining key insights. 2) Voice Transcription (via Whisper): Converts lectures into structured notes with over 85% accuracy. 3) Repeated Question Detection: Enables focused preparation by surfacing questions that appear across materials.

# Chapter 2

# Literature Review

The integration of artificial intelligence into education has seen rapid development, particularly in enhancing personalized learning experiences and automating academic workflows. Several studies have explored the relevance and application of AI in note-taking, voice recognition, document processing, and educational content generation. The following research works highlight the core ideas and technologies that inspired and shaped the development of the SMNS - Smart Multi-Note System.

## 2.1 Handwriting Recognition in Digital Note Taking

This study presents a unified framework that merges text recognition with generative intelligence for automated document processing, focusing on accurate extraction, summarization, and contextual interpretation of textual data. It incorporates preprocessing techniques like noise reduction and layout analysis to enhance recognition accuracy, while leveraging advanced language processing to ensure meaningful and semantically rich outputs. By addressing challenges such as unstructured formats and distorted input quality, the framework facilitates seamless conversion and transformation of raw content into structured, insightful summaries across domains like education, legal, and administration. [1]

The methodologies outlined in this work directly support core functionalities of the Smart Multi-Note System, such as handwriting-to-text conversion, PDF note extraction, and dynamic lecture summarization. The integrated approach enhances SMNS's ability to handle diverse content sources—handwritten,

printed, or spoken—and convert them into actionable formats like FAQs, summaries. Emphasis on preprocessing and semantic validation strengthens the system's adaptability and reliability in academic environments.

## 2.2 Whisper: Robust Speech Recognition System

Whisper is a multilingual speech recognition model developed by OpenAI. Trained on an extensive dataset of 680,000 hours of supervised audio, it boasts remarkable performance in speech transcription, translation, and detection tasks. The architecture is based on a sequence-to-sequence Transformer, enabling it to transcribe speech with high accuracy even in noisy environments and across diverse accents and languages. Unlike traditional ASR systems that struggle with variability, Whisper's robustness makes it suitable for real-time transcription and multi-modal understanding. [2]

This model directly informs the lecture voice analyzer and audio summarization feature of SMNS. Its ability to handle noisy, spontaneous, and accented speech aligns with classroom conditions, ensuring accurate transcription of lectures or voice notes. Integration of Whisper enables SMNS to convert spoken academic content into structured text, summarize it, and even answer queries based on it, making it an indispensable AI backbone for auditory learning modules.

## 2.3 AI in Education: Personalized Learning Assistants

This paper presents a systematic review of AI applications in higher education from 2007 to 2018, encompassing 146 empirical studies. The research categorizes AI's roles into four domains: learning support, educational management, prediction and profiling, and ethical considerations. It explores how AI enhances personalized learning, automates administrative tasks, predicts student performance, and introduces novel challenges like algorithmic bias and data privacy. The review provides a consolidated view of the current capabilities and limitations of AI in academia.[3]

The review provides critical theoretical grounding for the educational pur-

pose and ethical design of SMNS. It validates the use of AI-powered tools to personalize content delivery, support academic performance, and streamline student workflows. Additionally, it emphasizes the importance of transparency and ethical AI practices, which is crucial for SMNS, especially in modules involving sensitive student data like quiz performance or document analysis.

## 2.4 Extractive and Abstractive Summarization Techniques

This study presents a smart system that combines text recognition with AI-based content generation to make document reading and understanding easier. It improves the normal OCR (Optical Character Recognition) process by not just converting images or handwriting into text, but also summarizing the content and giving useful insights. Techniques like noise removal and page layout analysis help improve the accuracy of recognition. After that, language models make sure the final output is meaningful and relevant. This method solves common problems like unstructured documents and confusing formats, and it also deals with poor input quality and errors in AI-generated text. The system is useful in areas like education, office work, and legal documentation by turning long or messy text into clear and summarized information. [4]

This approach is very important for how the Smart Multi-Note System (SMNS) works. SMNS uses similar ideas to turn handwriting, printed notes, PDFs, and spoken lectures into clean digital text and helpful summaries. This helps students quickly understand and revise topics. It also supports features like creating FAQs, short notes. The focus on cleaning the input and checking the output helps SMNS work better in real-world learning situations and gives users more accurate and helpful results.

## 2.5 Multimodal Intelligence via Gemini API

Gemini is Google DeepMind's latest large multimodal model (LMM) platform, integrating capabilities across text, images, audio, and code. It's designed to offer seamless API access to powerful LLMs that can interpret and generate responses across multiple input types. With enhanced reasoning,

retrieval-augmented generation (RAG), and fine-tuned safety mechanisms, Gemini is positioned as a general-purpose cognitive agent for developers building AI-rich applications. [5]

Relevance to SMNS: Gemini's multimodal capabilities offer a scalable architectural blueprint for SMNS's evolution. For example, integrating Gemini can allow the system to accept handwritten PDFs, spoken lectures, and typed notes all in one query to generate contextual summaries, quizzes. Its flexible API model and advanced LLM capabilities align with SMNS's long-term vision of becoming a full-stack, AI-augmented academic assistant.

Together, these studies form the technological and theoretical foundation of the Smart Multi-Note System (SMNS), influencing its core functionalities—handwriting recognition, audio processing, summarization, and intelligent academic support. The integration of Optical Character Recognition (OCR) with Large Language Models (LLMs) enables accurate extraction and semantic understanding of handwritten and printed content. Speech recognition models like Whisper empower real-time transcription of lectures, while abstractive summarization techniques facilitate efficient content condensation and FAQ generation. Furthermore, multimodal frameworks like Google's Gemini offer scalable support for diverse inputs—text, audio, and images—aligning with SMNS's goal of becoming a unified academic assistant. By merging these technologies with ethical and pedagogical principles, SMNS transcends automation, delivering personalized, AI-enhanced learning experiences that redefine how students capture, organize, and engage with educational content.

# Chapter 3

# System Study

The current landscape of digital note-taking tools offers limited support for higher-order learning tasks such as extracting repeated questions, summarizing content, or generating quizzes. Most existing solutions focus solely on capturing notes rather than transforming them into active learning materials. Through extensive observation and student feedback, we identified the need for an integrated AI-powered platform that could bridge this gap. The SMNS Smart Multi-Note System aims to provide a complete ecosystem that not only stores notes but also enriches them through AI-generated insights, summaries, and visualizations. This system study outlines the limitations of existing tools and how our proposed system addresses them.

## 3.1 Existing System

Most current note-taking apps, such as Google Keep, Evernote, or OneNote, allow basic features like typing, drawing, and syncing across devices. However, these platforms: Lack real-time AI processing capabilities for handwritten input. Do not support smart analysis of PDFs or lecture recordings. Provide no academic assistance like Q&A generation, repeated question detection. Offer limited or no integration with LLMs for summarizing, extracting, or generating educational content. These systems are primarily repositories of notes rather than intelligent tools for learning and retention.

## 3.2 Proposed System

The SMNS – Smart Multi-Note System – is an end-to-end AI-powered platform designed to enhance student learning through intelligent note inter-

action. It incorporates handwriting recognition, audio transcription, NLP, and generative AI to deliver a personalized academic assistant.

Core Advantages: Real-time handwriting-to-font conversion and storage. Voice processing to extract key concepts from recorded lectures. Smart PDF processing to identify repeated exam questions. Summarization, extraction, and quiz generation. AI-powered Q&A interface for concept clarification. These functionalities are modular, enabling future scalability and easier maintenance. The app is built using a cross-platform stack to ensure broad accessibility.

## 3.3 Software Requirements Specification

The system is designed to empower students with a set of tools that go beyond traditional, passive note-taking. By transforming static notes into interactive, structured, and revision-friendly formats, the Smart Multi-Note System (SMNS) enhances comprehension and promotes deeper engagement with learning materials. The aim is to support a more personalized and intelligent study workflow, particularly suited for academic environments.

The SMNS application is tailored to meet the needs of college students across a wide range of disciplines. It provides multiple modes of input including handwriting, voice recordings, and PDF uploads, allowing students to capture notes in the way that best suits their learning style. On the output side, the application delivers AI-generated text summaries, automatically extracted questions, and contextual answers. The platform features secure user management and cloud-based storage to ensure data safety and accessibility. At its core, the system integrates the powerful Gemini API, enabling advanced natural language understanding and response generation for features such as summarization, Q&A, and quiz creation.

### 3.3.1 Overall Description

The SMNS application operates as a self-contained platform, offering both

web and mobile interfaces. It provides seamless, real-time interaction between users and the AI-driven backend without requiring additional third-party platforms. However, in future versions, the app is expected to support integration with popular Learning Management Systems (LMS) such as Moodle or Google Classroom, thereby allowing the automatic import of course materials and shared academic resources.

### 3.3.2 Product Functions

The system encompasses several core functionalities designed to streamline the learning process through intelligent automation and enhanced user interaction. Users can upload notes in diverse formats such as handwritten sketches captured on a digital canvas, audio recordings from lectures or discussions, and academic materials in PDF format. These inputs are routed through dedicated pre-processing pipelines that standardize the data for further AI-driven analysis.

Handwritten content is transformed into digital text using advanced OCR techniques, with the added capability of rendering the output in a font style that closely replicates the user's natural handwriting. This not only aids in legibility but also preserves the personal touch of handwritten notes. For audio inputs, models like Whisper transcribe spoken words into structured text segments, ready for downstream tasks.The summarization module utilizes the Gemini API to distill extensive content into concise summaries, capturing key insights and themes to facilitate efficient review and comprehension. This functionality significantly reduces the cognitive load on students during revision by highlighting essential information.

For evaluation and engagement, the system includes a question extraction engine that analyzes uploaded notes and documents to identify recurring topics and generate quizzes. These quizzes include various formats such as multiple-choice questions and fill-in-the-blanks, enabling interactive self-assessment. To support dynamic learning, the platform features an AI-powered Q&A assistant that responds to user queries with contextual understanding. This chatbot interprets the content of previously uploaded materials to deliver relevant and accurate responses, fostering an interactive and

personalized learning experience.

## 3.4 Software Requirements Specification

This chapter outlines both the functional and non-functional requirements of the SMNS - Smart Multi-Note System. It includes details on system interfaces, performance expectations, and architectural decisions that guide the development of the application. This section outlines the detailed technical requirements of the Smart Multi-Note System (SMNS), covering both functional and non-functional aspects. It defines the various external interfaces the system interacts with, the expected features and capabilities from a user perspective, and the performance, usability, and scalability standards the system must meet. These requirements serve as the foundation for development, implementation, and validation of the application.

### 3.4.1 External Interfaces

The SMNS (Smart Multi-Note System) establishes robust connectivity with users, hardware, and software through a set of well-defined external interfaces. These interfaces facilitate seamless interaction between the application's frontend, backend services, and supported devices. Adhering to contemporary design standards, the system ensures reliable operation across various platforms. The following outlines the key aspects of the user interface, API integration, hardware requirements, and software compatibility. The application features a responsive user interface compatible with mobile, tablet, and desktop devices. Users can input notes via typing, handwriting, audio uploads, or PDFs. The system processes this input and presents summaries, questions, through interactive dashboards. Secure login with role-based access ensures that sensitive actions like data deletion and AI interaction are limited to authorized users.

The system's frontend, built with React Native, communicates with the backend services and AI modules through well-defined RESTful APIs developed using Flask. These APIs handle all data transactions, including audio transcription via the Whisper model and natural language processing tasks using the Gemini API. The API layer is modular and extensible, allowing for seamless integration with future services such as Learning Management

Systems (LMS) or third-party cloud storage providers.

The SMNS system is compatible with a wide range of hardware configurations. Devices must support HTML5 Canvas for handwriting input, access to microphones for voice recordings, and the ability to upload files. The minimum hardware requirements include 2GB of RAM, basic internet connectivity, and audio input support.The application is optimized for major web browsers such as Google Chrome, Safari, Mozilla Firefox, and Microsoft Edge. It integrates with MongoDB as the primary data storage solution, ensuring efficient handling of both raw user inputs and AI-generated outputs.

### 3.4.2 Functional Requirements

The functional capabilities of the SMNS application are designed to facilitate intelligent note processing and user interaction. The system accommodates multiple input formats including typed text, handwritten notes, audio files, and PDFs and employs advanced AI models to process and generate structured outputs such as summaries, questions. These features collectively contribute to an efficient and adaptive digital note-taking experience suited for academic environments. The system allows users to upload and input notes in various formats. Handwriting can be captured using touchscreen or drawing input methods. Users can also upload audio files in formats such as WAV and MP3, which are processed for transcription. PDF documents containing class notes or question papers can be uploaded for analysis, and there is also an option for manually typing notes directly into the application.

Once input is received, the AI module takes over with various processing pipelines. Handwritten text is converted into digital format using OCR and mapped to the user's personalized font. Audio files are transcribed using the Whisper model or an equivalent speech-to-text engine. The large language model (LLM), powered by Gemini, is responsible for generating content-specific outputs including concise summaries, quiz questions, and contextual answers. Additionally, the system detects and extracts repeated questions from uploaded materials to aid in exam preparation. The user interface provides rich visualizations and real-time feedback. Processed content such as summaries and questions are displayed in intuitive dashboards. Users

can interact with an AI Q&A chatbot to clarify concepts.

### 3.4.3 Non-Functional Requirements

Beyond core functionalities, SMNS is designed with a strong focus on system quality attributes that enhance user experience and long-term sustainability. Key considerations include performance responsiveness, intuitive usability, fault tolerance, and ease of maintenance. The architecture also supports scalability for growing user demands and ensures portability across platforms to provide a consistent experience on both web and mobile environments. The system is optimized for quick responsiveness. User interface actions are expected to complete in less than two seconds, while AI processing tasks, including transcription and summarization, should return results within five seconds. The platform is designed to support over 100 concurrent users without noticeable degradation in performance.

SMNS prioritizes accessibility and ease of use. The interface is simple and clean, designed to cater to both technical and non-technical users. Tutorials and tooltips are embedded within the app to assist first-time users with navigation and feature discovery, ensuring minimal onboarding effort. The system includes mechanisms for auto-saving notes and recovering unsaved data in the event of an application crash. Its modular architecture allows developers to implement updates, improvements, and new features with minimal disruption. Rigorous unit and integration testing are performed prior to deployment to ensure system stability and accuracy.

To accommodate future growth, the system is built with scalability in mind. Its architecture supports horizontal scaling through containerization technologies such as Docker and Kubernetes. Backend services are deployable on cloud platforms like Amazon Web Services (AWS) or Google Cloud Platform (GCP) to handle increased load as user adoption grows. The SMNS system is available as a Progressive Web App (PWA), and a Android App enabling cross-platform compatibility and offline access. The use of React JS with TypeScript ensures a maintainable codebase with reusable components and strong type safety, facilitating portability and long-term development.

# Chapter 4

# System Design

The system employs a modular client-server architecture for scalability, maintainability, and efficient data exchange. The client layer captures input types such as text, handwriting, audio, and PDFs, which are then sent to the backend for processing, ensuring a flexible and accessible user experience across devices. *Refer fig: 4.1 System Architecture of SMNS.*

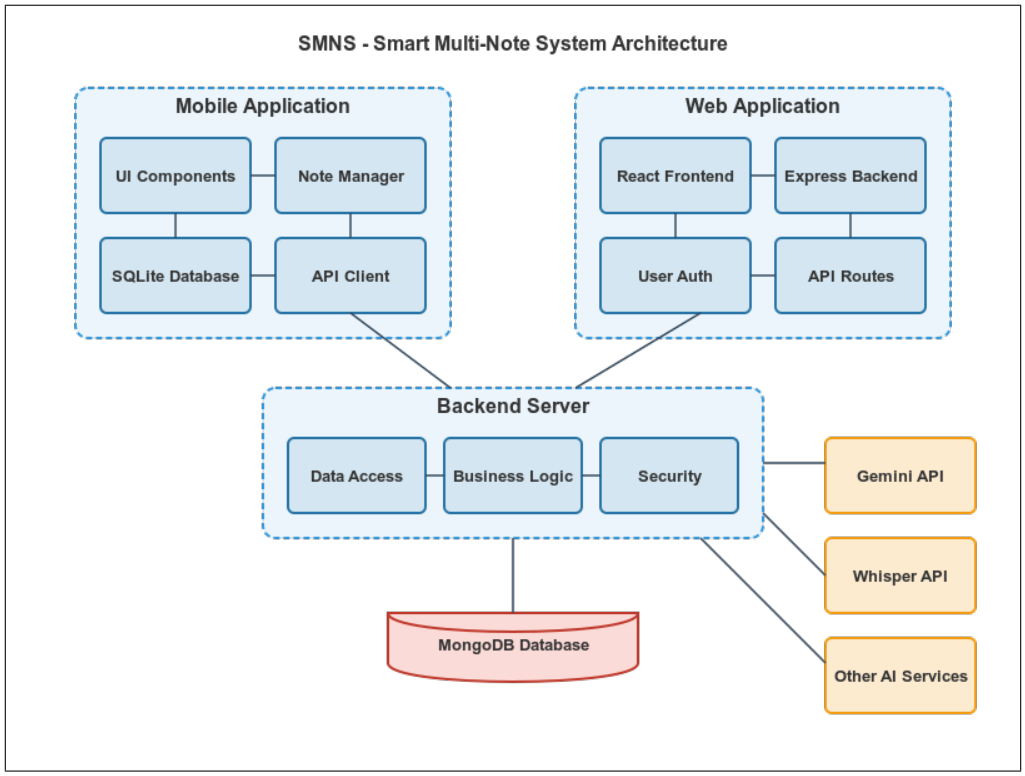



Figure 4.1: System Architecture of SMNS

Main Components:

The system consists of three primary layers: Input, Processing, and Output. The Input Layer includes a handwriting canvas, voice recorder, and PDF uploader, allowing users to submit data in multiple formats. The Processing Layer leverages AI models like Whisper and the Gemini API, along with a Flask backend and MongoDB for processing and storage. The Output Layer generates intelligent responses such as summaries, extracted questions, and a Q&A chatbot, providing users with structured, actionable insights.

On the server side, the system manages key functions such as user authentication, AI-driven content extraction, secure data storage, and cross-platform synchronization. Advanced AI models and NLP modules process the input to generate structured outputs like summaries, quiz questions, which are returned to the client for interaction. Secure data storage ensures that sensitive information is protected, while synchronization keeps data consistent across multiple devices. The system also incorporates role-based access control (RBAC), restricting critical operations such as data deletion and user management to administrative users, enhancing security.

Overall, the architecture is built to ensure performance, security, and flexibility, allowing the system to evolve while maintaining seamless user interaction.

## 4.1 Data Flow Diagram (DFD)

The DFD illustrates the interaction between users (students/admins), input modules (handwriting, voice, PDFs), the AI processing engine, and output modules (summaries & questions). It captures how user data flows through the system, is processed by AI, and transformed into meaningful outputs. *Refer fig: 4.2 Dataflow Diagram Of SMNS.*

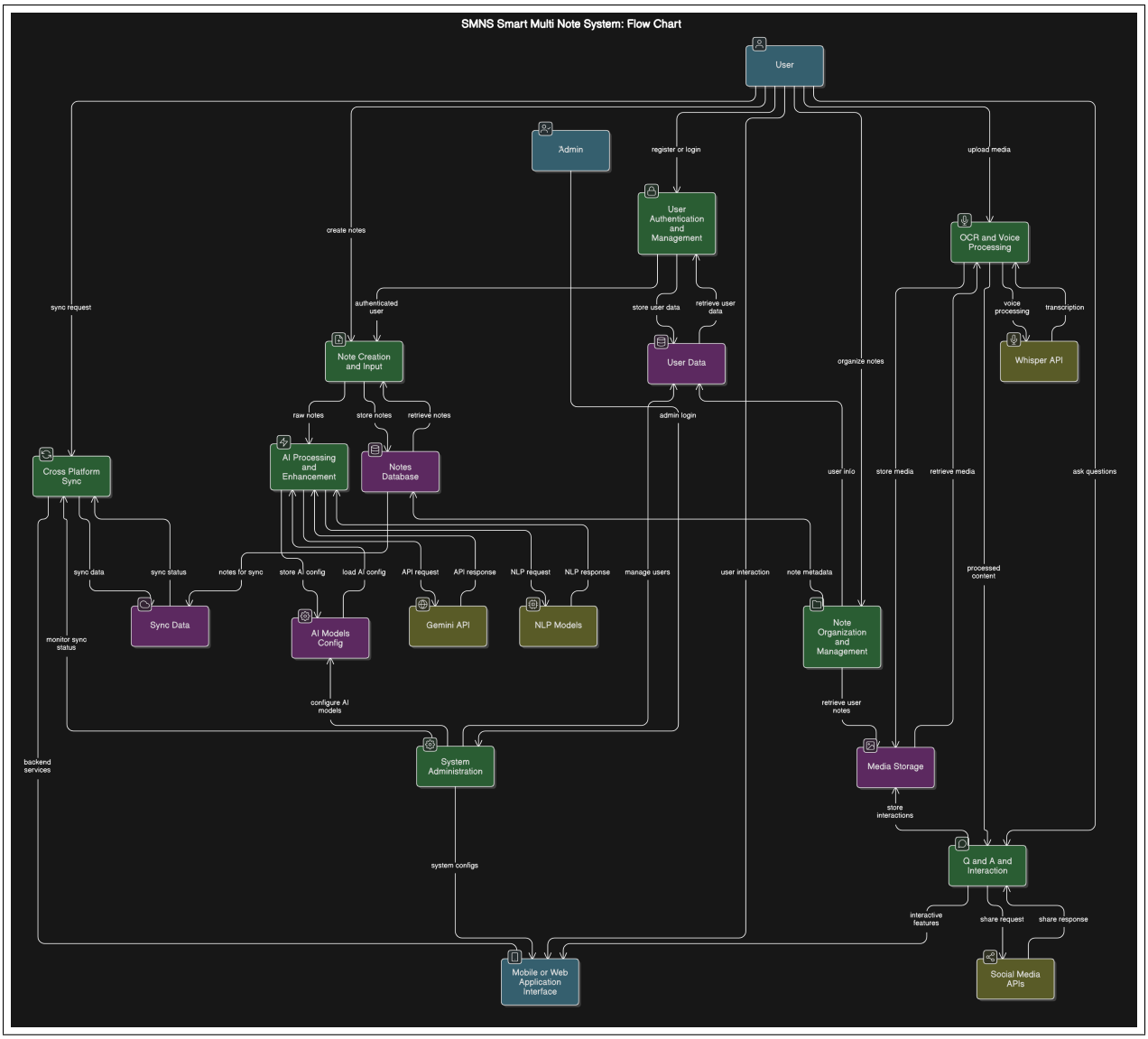



Figure 4.2: Dataflow Diagram Of SMNS

## 4.2 Use Case Diagram

The Use Case Diagram highlights the different interactions between users (actors) and the system (use cases). It depicts three user roles: User, Approver, and Admin. Users can create, generate, and track applications. Approvers can approve or reject applications, while Admins register users. *Refer fig: 4.3 Use Case Diagram of the System.*

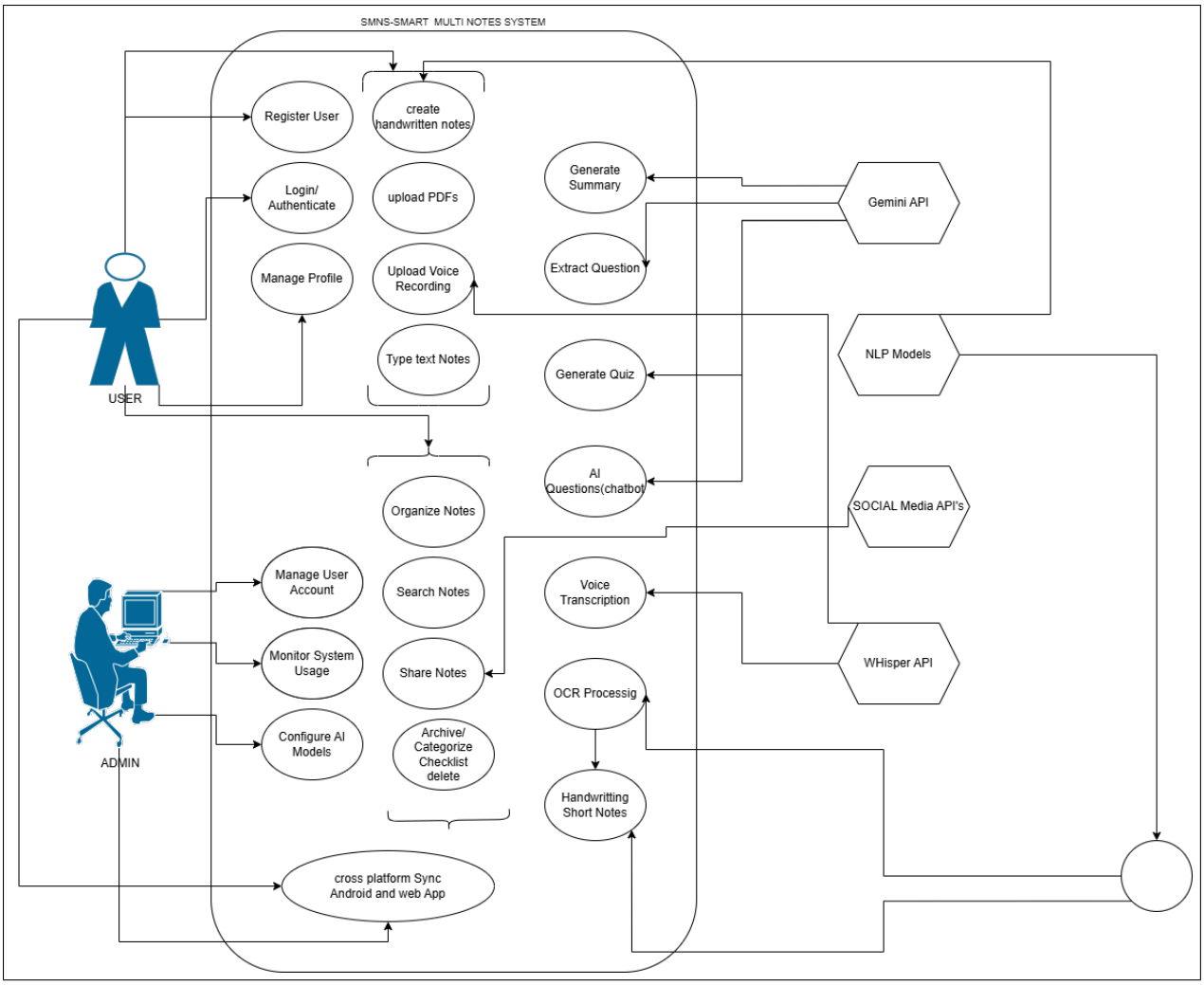



Figure 4.3: Use Case Diagram of the System

## 4.3 Database Design

The database design for the system revolves around two primary entities: User and Note. These entities are structured to support the core functionalities of the system, such as managing user accounts, processing different input types, and storing processed outputs. The User entity holds essential information about the system users, whether they are students or administrators, while the Note entity manages the content created by users, including text, voice recordings, handwriting, and PDFs. This organized database structure facilitates efficient querying, data retrieval, and scalability as the system grows. *Refer to Table 4.1: User Schema & able 4.2: Note Schema.*

### User Schema

Table 4.1: User Schema: Properties, Descriptions, and Data Types

| Property Name | Description | Data Type |
|---|---|---|
| userId | Unique ID | String |
| name | Full name | String |
| email | Email address | String |
| password | Encrypted password | String |
| role | 'student' or 'admin' | Enum |
| createdAt | Timestamp | Date |

### Note Schema

Table 4.2: Note Schema: Property Name, Descriptions, and Data Types

| Property Name | Description | Data Type |
|---|---|---|
| noteId | Unique ID | String |
| ownerId | Reference to User | ObjectId |
| inputType | 'text', 'voice', 'handwriting', 'pdf' | Enum |
| rawInput | File or raw content | Mixed |
| aiOutput | JSON containing summary, questions | Object |
| dateUploaded | Timestamp | Date |

## 4.4 Model and Dataset

The Gemini model is fine-tuned using a dataset of over 300 student notes and formal questions, which include academic writing and class notes. This model is employed for tasks such as summarization, question generation, and Q&A, enabling it to generate relevant, context-aware outputs for educational use. By leveraging this dataset, the Gemini model ensures that the processed content is structured and applicable to academic contexts.

The Whisper model processes uploaded lecture audio by transcribing it and segmenting the voice content into usable chunks. This functionality is essential for converting spoken content, such as lectures, into structured text format, allowing students to capture and review lecture material in a readable form. Whisper ensures that audio-based inputs are accurately transcribed and accessible for further analysis. Fine-tuning of the models is achieved by using structured prompts with the Gemini API. These prompts are specifically tailored to generate summaries, and Q&A based on student notes and academic materials. The fine-tuning process ensures that the AI outputs

align with educational needs, delivering relevant and accurate content for students and educators. This method guarantees that the system provides context-specific, high-quality outputs.

## 4.5 Input-Output Design

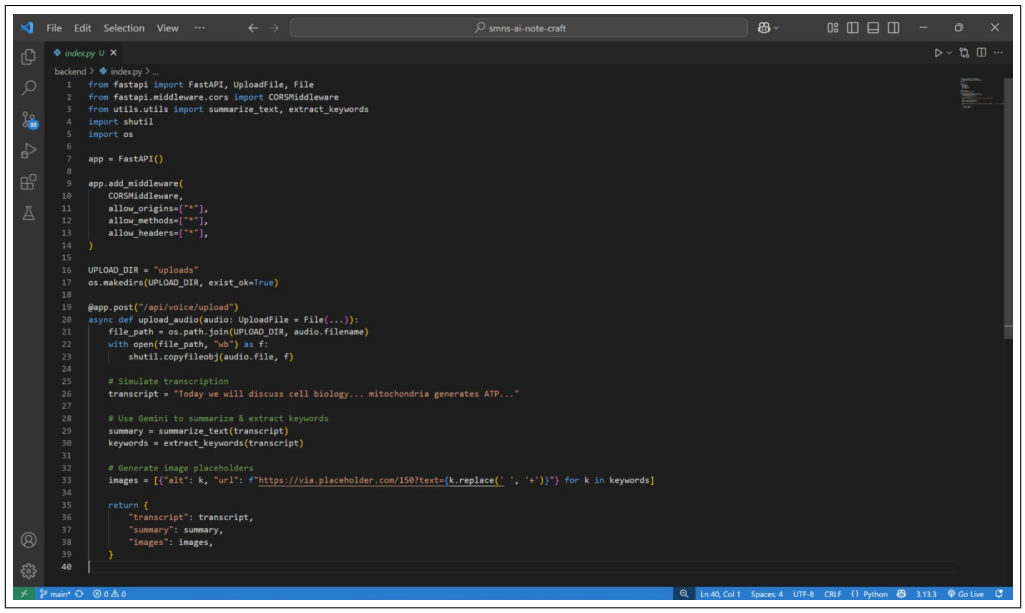
The Input-Output Design focuses on providing an intuitive and highly functional interface for seamless interaction between the user and the underlying AI-powered backend systems. Each input mechanism is tailored to support multiple types of data, while the outputs are designed to maximize comprehension, productivity, and learning effectiveness. The goal is to accommodate varied user preferences be it voice, text, handwriting, or document-based inputs and transform them into structured, intelligent, and usable academic content. This system architecture is designed to operate consistently across both mobile and web platforms, leveraging cloud-based services to ensure synchronization, accessibility, and performance.

### 4.5.1 User Registration

During the registration process, users are required to provide their email, password, and name. Upon successful submission and validation of these details, the system securely stores the information and redirects the user to their personalized dashboard. This registration process initiates the user's journey within the application and enables access to all other modules.

### 4.5.2 Voice Upload

The voice upload feature allows users to submit audio recordings, such as lecture clips or spoken notes, as input. These audio files are processed by the Whisper model or an equivalent transcription engine, which converts them into accurate textual transcriptions. The resulting text serves as a base for additional features like summarization or question generation. *Refer fig: 4.4 Data Upload Block.*



```python
from fastapi import FastAPI, UploadFile, File
from fastapi.middleware.cors import CORSMiddleware
from utils.utils import summarize_text, extract_keywords
import shutil
import os

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

UPLOAD_DIR = "uploads"
os.makedirs(UPLOAD_DIR, exist_ok=True)

@app.post("/api/voice/upload")
async def upload_audio(audio: UploadFile = File(...)):
    file_path = os.path.join(UPLOAD_DIR, audio.filename)
    with open(file_path, "wb") as f:
        shutil.copyfileobj(audio.file, f)

    # Simulate transcription
    transcript = "Today we will discuss cell biology... mitochondria generates ATP..."

    # Use Gemini to summarize & extract keywords
    summary = summarize_text(transcript)
    keywords = extract_keywords(transcript)

    # Generate image placeholders
    images = [{"alt": k, "url": f"https://via.placeholder.com/150?text={k.replace(' ', '+')}"} for k in keywords]

    return {
        "transcript": transcript,
        "summary": summary,
        "images": images,
    }
```

Figure 4.4: Data Upload Block

### 4.5.3 PDF Upload

Through the PDF upload module, users can submit academic documents such as question papers or handwritten notes in digital format. Once the PDF document is uploaded, the backend employs NLP techniques to analyze the content and extract repeated questions. These questions are then displayed in a user-friendly format for easy reference and focused exam preparation. *Refer fig: 4.4 Data Upload Block.*

### 4.5.4 Handwriting Input

For users who prefer writing by hand, the system includes a handwriting canvas. Users can input text using a stylus or finger directly on the screen. The system captures the handwritten strokes and applies OCR or handwriting recognition to convert them into text rendered in the user's personalized handwriting font. This creates a familiar yet digital note-taking experience. *Refer fig: 4.5 Backend Python Snippet.*

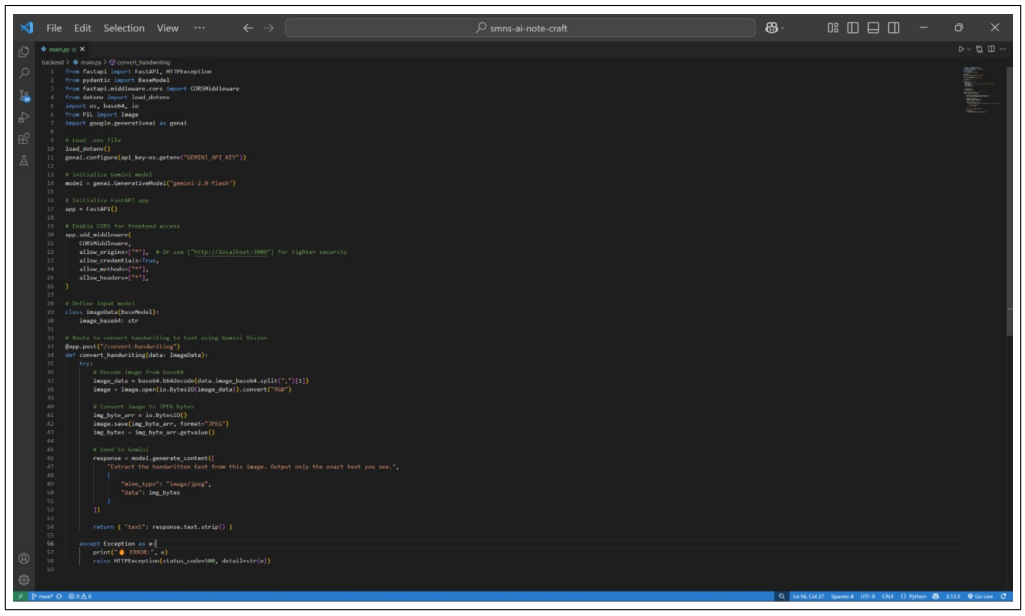



Figure 4.5: Backend Python Snippet

### 4.5.5 AI Summary Generator

Users can submit raw text from lectures, handwritten notes, or transcriptions into the summarization module. The raw input notes are analyzed by the Gemini API, which condenses them into concise, well-structured summarized paragraphs. This feature enhances learning efficiency by enabling quick reviews and better content retention. *Refer fig: 4.6 API custom prompting.*

### 4.5.6 AI Q&A Interface

The question-answering interface allows users to input custom academic queries along with contextual notes. This module accepts a combination of the user's question and relevant content as input, processes it using an LLM, and returns a contextually accurate and concise answer. It serves as an AI tutor, helping students clarify concepts in real-time.

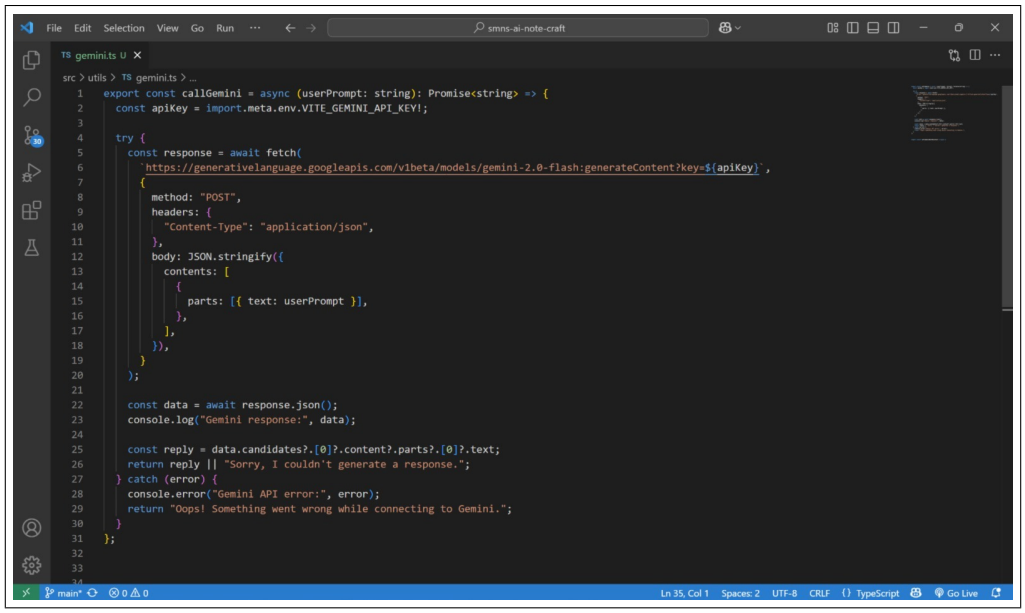



Figure 4.6: API custom prompting

## 4.6 Modular Design

The User Authentication module is responsible for managing user registration and login through JSON Web Tokens (JWT). It differentiates between student and admin roles to ensure secure, role-based access to system features. By validating credentials and issuing encrypted tokens, it guarantees that only authorized users can access or modify protected resources. The Note Manager module facilitates the uploading and storage of diverse input types, including PDFs, audio recordings, and handwritten notes. It categorizes and organizes notes efficiently within the database, ensuring seamless retrieval and further AI-driven processing. *Refer fig: 4.4 Data Upload Block; 4.5 Backend Python Snippet 4.6 API Custom Prompting.*

The AI Engine acts as the core processing unit that orchestrates note interpretation using large language models (LLMs) such as Gemini and Whisper. It handles advanced tasks including summarization, QA generation, and transforming raw inputs into structured note content, enabling intelligent content understanding and transformation. This module supports real-time lecture transcription, automatically identifying key discussion points and condensing spoken input into concise summaries. Additionally, it enables automatic quiz generation by analyzing the document context, aiding in exam

preparation. Personalized feedback mechanisms and intelligent highlighting of frequently asked concepts ensure a more tailored and effective study experience.

The Frontend Renderer module is responsible for rendering the user interface, displaying dashboards and processed outputs. It provides a clean, interactive experience across platforms, ensuring smooth navigation and visualization of AI-generated content. Features include live note previews, responsive search filtering, and access to custom font-rendered outputs that preserve users' original handwriting styles for a familiar and personalized touch.

# Chapter 5

# System Implementation

The SMNS – Smart Multi-Note System has been developed using a modular and scalable architecture that ensures robustness and flexibility in deployment. The system is designed with a clear separation of concerns, wherein the frontend and backend components are hosted on independent servers. These components communicate through secure RESTful API endpoints established in the backend, enabling a reliable and efficient exchange of data. All interactions between the frontend and backend are conducted in JSON format, which facilitates smooth data serialization, seamless integration, and consistent rendering of user-friendly interfaces.

The implementation strategy adhered to a structured Software Development Life Cycle (SDLC), beginning with a thorough requirement analysis phase that involved identifying the core features and user expectations. This was followed by a modular design approach, ensuring that each functionality was independently scalable and maintainable. Feature development was carried out in incremental stages, allowing for continuous improvement and refinement of each module. Once the features were developed, rigorous integration and testing were conducted to validate the interoperability and correctness of all components. Finally, the system was deployed in a staged manner, followed by iterative feedback collection to optimize performance, user experience, and reliability.This structured approach not only streamlined the development process but also laid the foundation for efficient maintenance and future upgrades.

## 5.1 Modular Implementation

The Smart Multi-Note System was implemented using a modular architecture, with each component handling a distinct part of the overall workflow. This modularization enables better scalability, testing, and maintainability across the stack. The project was broadly divided into four major components: frontend development, backend development, database integration, and AI module integration.

### 5.1.1 Frontend Development (React JS + TypeScript)

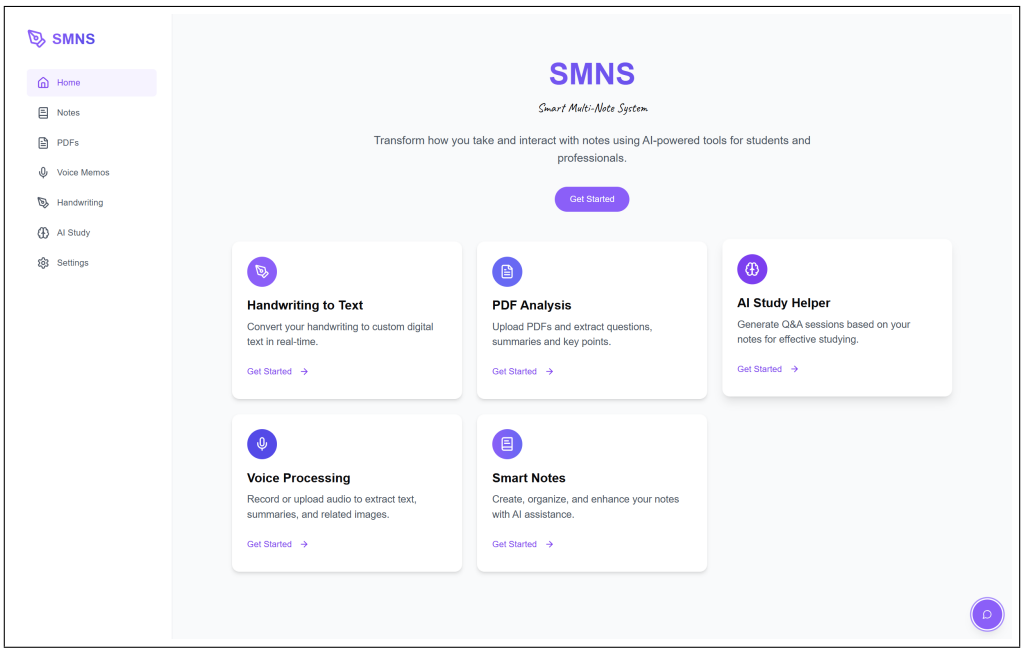



Figure 5.1: Home page of Web App

The frontend of the Smart Multi-Note System (SMNS) is developed using React.js, which is chosen for its efficient component-based structure and fast rendering. TypeScript is also used to add static type-checking, which helps improve code maintenance and reduces the chances of errors. For a consistent and responsive design, CSS Modules and reusable UI components are used. This method helps keep the code clean, ensures modular styling, and guarantees the same user interface behavior across different devices. *Refer fig: 5.1 Home Page of Web App.*

The home page is the central hub of the application, designed to provide users with a seamless and efficient interface for accessing all the key features of the Smart Multi-Note System (SMNS). Whether accessed via the web or Android app, the layout is clean, intuitive, and user-friendly, ensuring that users can navigate through the application effortlessly. The page is built with responsiveness in mind, meaning that it automatically adapts to different screen sizes and resolutions, whether the user is on a smartphone, tablet, or desktop. This responsive design ensures that the user experience remains consistent and optimal across a wide range of devices, preventing any functionality issues due to screen size variations. *Refer fig: 5.2 Home Page of Android App.*

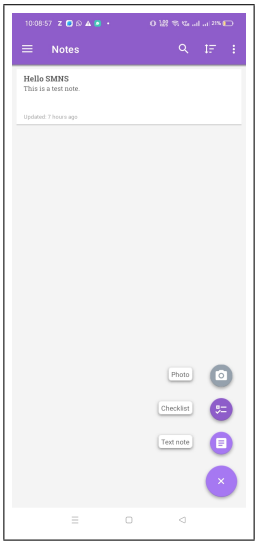



Figure 5.2: Home page of Android App

One of the standout features of the home page is the canvas-based handwriting input, which allows users to write directly on the screen using a stylus or touch-enabled devices. This input method is designed to replicate the traditional experience of writing on paper, providing users with a natural and intuitive way to take notes. The canvas is specially optimized for touch interactions, offering smooth and precise drawing and writing capabilities. This feature is ideal for users who prefer to write by hand rather than typing, making the note-taking process feel more personal and flexible. Whether jotting down quick thoughts, drawing diagrams, or writing detailed notes, the handwriting input offers a seamless and efficient method for capturing ideas in a way that is both easy to use and responsive.

The handwriting input feature utilizes OCR (Optical Character Recognition) technology and a FastAPI-based backend for seamless conversion of handwritten notes into digital text. When a user writes on the canvas, the drawing is saved as a PNG image. This image is then sent to the backend through a FastAPI endpoint. The server processes the image using OCR algorithms, which detect and extract the handwritten characters. The resulting text is sent back to the frontend, allowing users to view and edit their handwritten notes in digital format. This ensures a fast, accurate, and user-friendly note digitization process. *Refer fig: 5.3 Hand Canvas.*

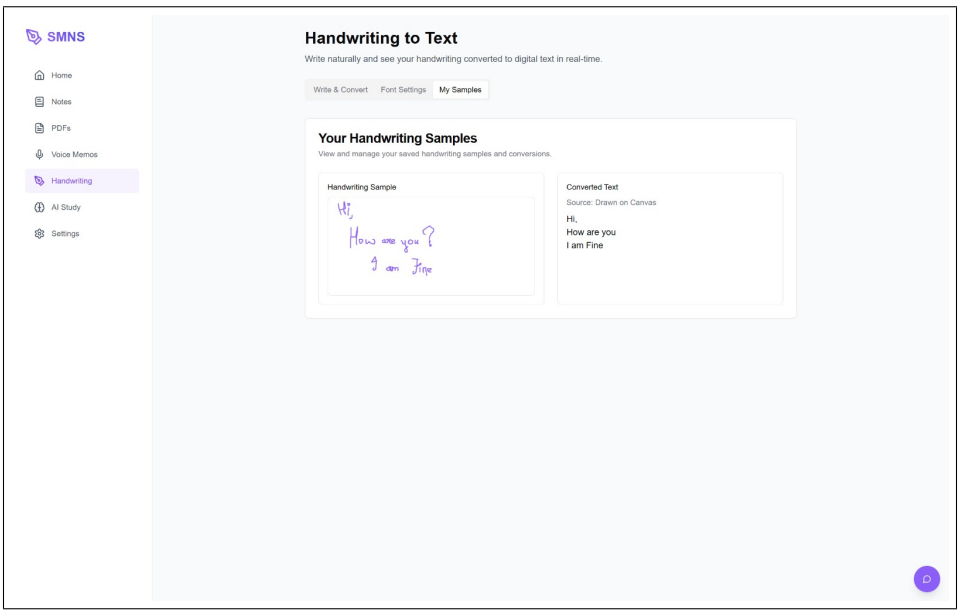



Figure 5.3: Hand Canvas

In addition, the system supports voice-based input through integrated audio recording and file upload functionality, enabling seamless incorporation of voice notes. For traditional document-based workflows, a file picker is provided to facilitate the direct import of PDFs and text files, streamlining the process of transitioning from physical to digital note-taking.

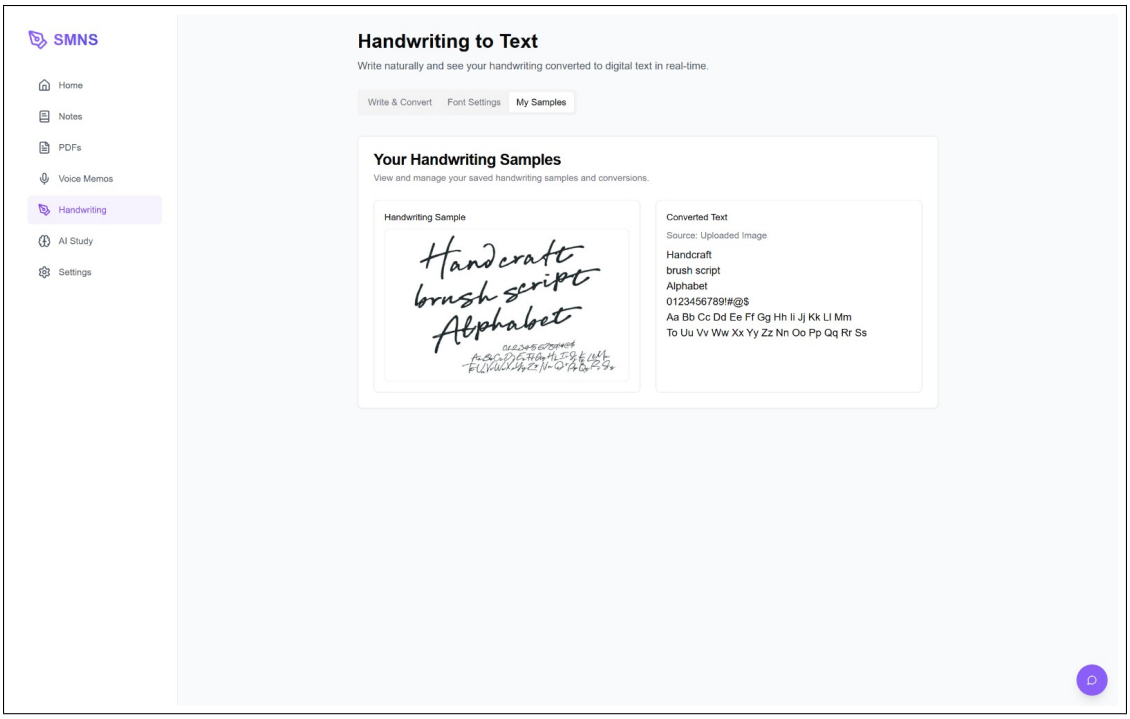



Figure 5.4: Modes Of Data to Be Uploaded

Users can upload PDFs through a simple file picker interface, making it easy to import notes, scanned pages, or other documents. For text-based PDFs, the system extracts the content directly. For scanned or handwritten PDFs, it uses OCR technology to analyze and convert even complex human handwriting into editable digital text. *Refer fig: 5.4 Modes Of Data to Be Uploaded.*

The backend is powered by Flask, which handles file processing and serves the extracted data to the frontend. This setup ensures smooth integration between the user interface and backend services, allowing users to view AI-enhanced outputs like summaries, quizzes, and Q&A within a responsive dashboard environment.

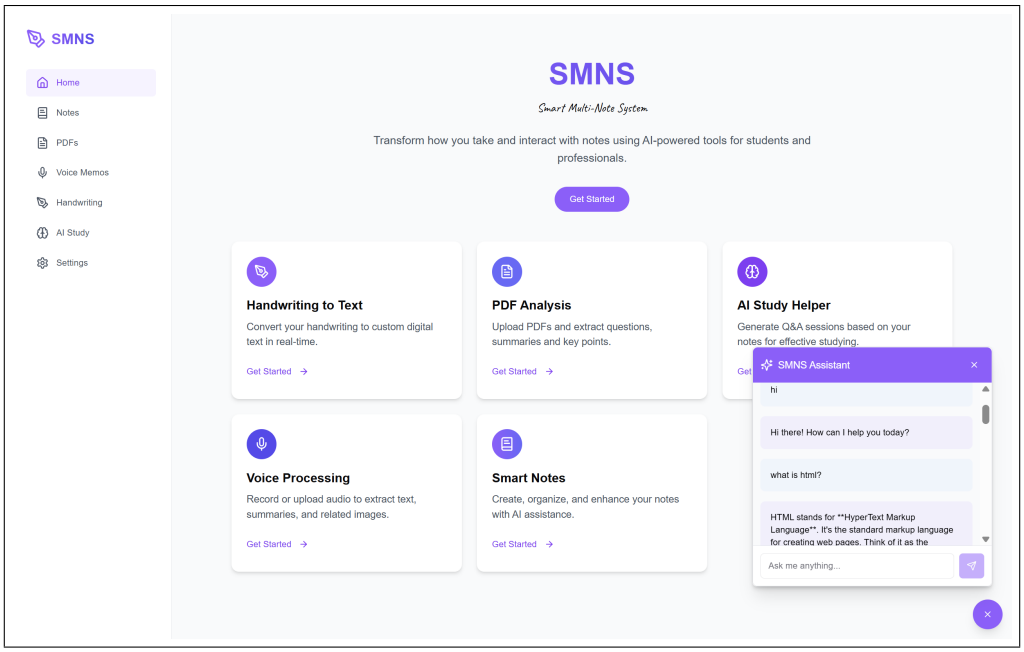



Figure 5.5: Chatbot

The chatbot feature allows users to interact with the system in a conversational manner. It can answer questions, provide summaries, and assist with navigating the app. This AI-powered assistant helps users get the information they need quickly and easily, enhancing the overall user experience. *Refer fig: 5.5 Chatbot.*

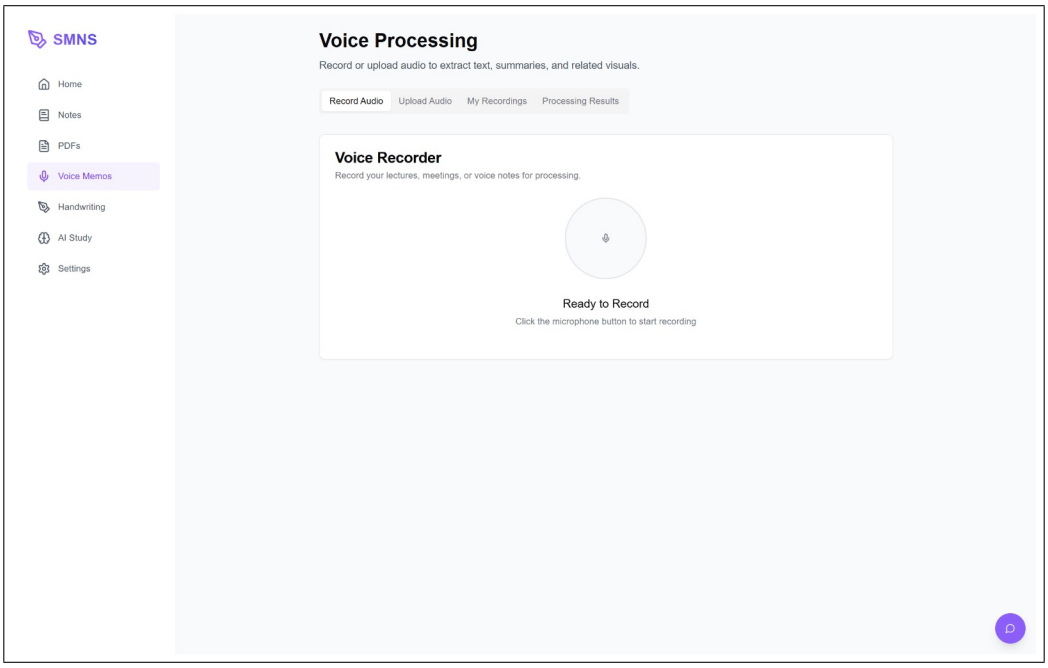



Figure 5.6: Voice

The Voice Transcription feature allows users to upload audio files directly from local storage for transcription. Once uploaded, the backend processes the file using speech-to-text technology to convert spoken content into written notes. While effective for clear recordings, the system performs best with clean, noise-free audio; heavily distorted or low-quality inputs may result in reduced transcription accuracy. This feature enhances accessibility and flexibility for users who prefer verbal input over typing or writing. *Refer fig: 5.6 Voice.*

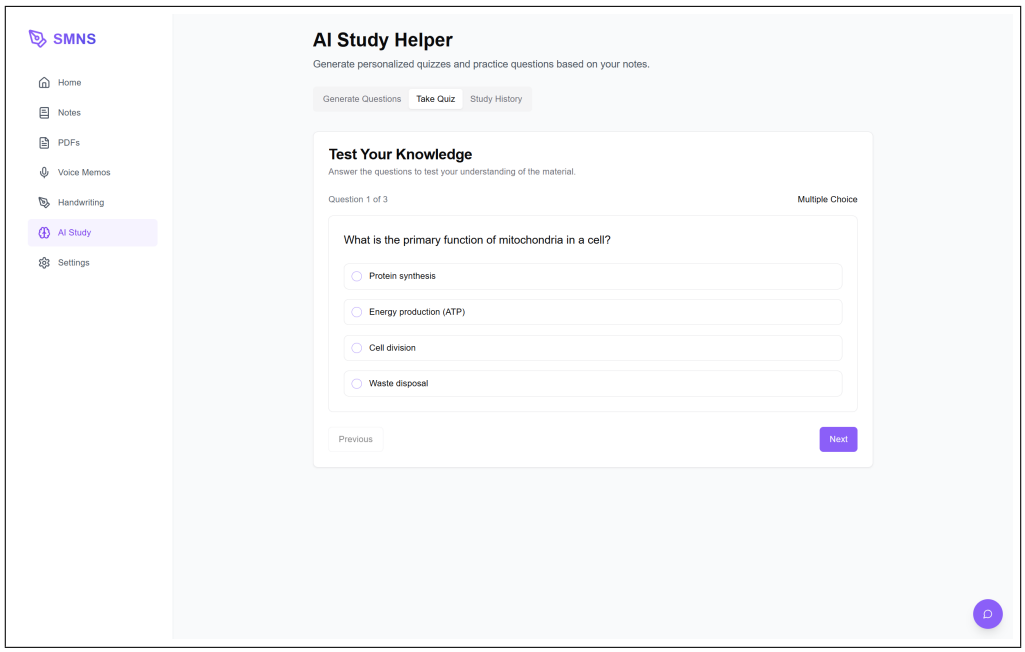



Figure 5.7: Quiz

The AI Quiz feature generates personalized questions based on summarized or extracted data from the user's input, including handwritten notes, PDFs, or typed content. After processing through OCR and Flask-based backend services, the content is analyzed to extract key points, facts, and definitions. Using this refined data, the system creates a diverse set of quiz questions such as multiple choice, fill-in-the-blanks, and short answers tailored to reinforce the user's understanding. This ensures that quizzes are both relevant and varied, aligning closely with the original learning material. *Refer fig: 5.7 Quiz.*

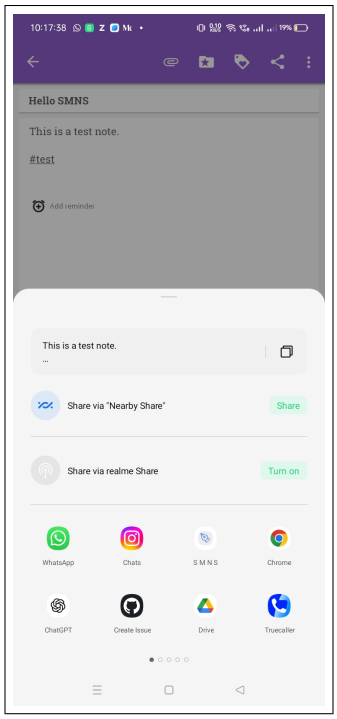


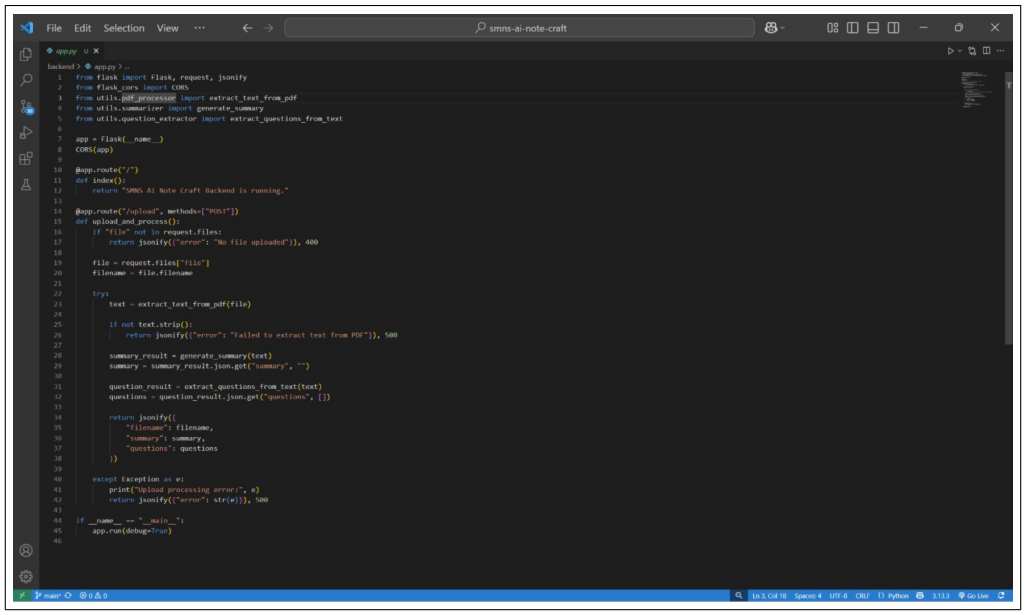

Figure 5.8: Ways to Share Notes

And, the system allows users to easily share their notes and other content. This sharing feature is designed to encourage collaboration among users. Whether they want to share with classmates, colleagues, or friends, users can seamlessly send their work. *Refer fig: 5.8 Ways to Share Notes.*

### 5.1.2 Backend Development (Flask)

The backend was developed using Flask, with RESTful APIs handling various application processes. Cross-origin requests are managed through Flask-CORS. For authentication, the Flask-JWT-Extended module is used to securely manage user login sessions. *Refer fig: 5.9 App Routing script.*

Middleware is added to check each uploaded file, make sure the user is allowed to access the system, and verify inputs before sending them to the main processing unit. This helps protect the system from errors and unwanted access. The backend also has separate paths for each task, such as converting audio to text, uploading notes, and saving AI-generated results like summaries. This clear structure makes it easier to update, manage, and expand the system in the future.



```python
from flask import Flask, request, jsonify
from flask_cors import CORS
from utils.pdf_processor import extract_text_from_pdf
from utils.summarizer import generate_summary
from utils.question_extractor import extract_questions_from_text

app = Flask(__name__)
CORS(app)

@app.route("/")
def index():
    return "SMNS AI Note Craft Backend is running."

@app.route("/upload", methods=["POST"])
def upload_and_process():
    if "file" not in request.files:
        return jsonify({"error": "No file uploaded"}), 400

    file = request.files["file"]
    filename = file.filename

    try:
        text = extract_text_from_pdf(file)

        if not text.strip():
            return jsonify({"error": "Failed to extract text from PDF"}), 500

        summary_result = generate_summary(text)
        summary = summary_result.json.get("summary", "")

        question_result = extract_questions_from_text(text)
        questions = question_result.json.get("questions", [])

        return jsonify({
            "filename": filename,
            "summary": summary,
            "questions": questions
        })

    except Exception as e:
        print("Upload processing error:", e)
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(debug=True)
```

Figure 5.9: App Routing script

### 5.1.3 Database Integration (MongoDB)

A cloud-hosted instance of MongoDB Atlas serves as the primary data storage solution, providing scalable and reliable management of diverse data types, such as user information, notes, AI-generated outputs, and transcribed audio. Data is organized into collections, ensuring logical separation and efficient access. Although MongoDB is schema-less, a Mongoose-like validation system enforces consistency by applying structured data rules, ensuring fields like user ID, date, and input type are indexed for quick querying. This approach combines MongoDB's flexibility with the necessary safeguards for data integrity.

### 5.1.4 AI Model Integration

The Smart Multi-Note System integrates multiple advanced AI models to enhance the user experience by automating core functionalities such as transcription, summarization, question generation, and handwriting recognition. These models are accessed through a modular backend, primarily built using Flask, which efficiently handles data flow between the frontend and AI processing layers. Audio inputs are transcribed using OpenAI's Whisper model, while PDF and handwritten content are processed using OCR

and NLP pipelines to extract structured text. Summarization, question answering, and quiz generation are driven by intelligent prompt-based logic, enabling the system to deliver high-quality, contextual educational content. The integration is designed to be scalable, accurate, and responsive across various input formats voice, handwriting, and documents ensuring a seamless user experience. *Refer fig: 5.10 Voice Processing.*

Audio files, are processed through a dedicated backend endpoint (/transcribe). This service leverages OpenAI's Whisper model a state-of-the-art speech recognition system capable of transcribing spoken language into coherent, paragraph-formatted text. The system performs exceptionally well in handling moderately clean audio input and is particularly valuable during lectures, meetings, or revision sessions where typing notes is impractical. Once the audio is transcribed, the output is formatted and stored, making it accessible for further processing like summarization and quiz generation.

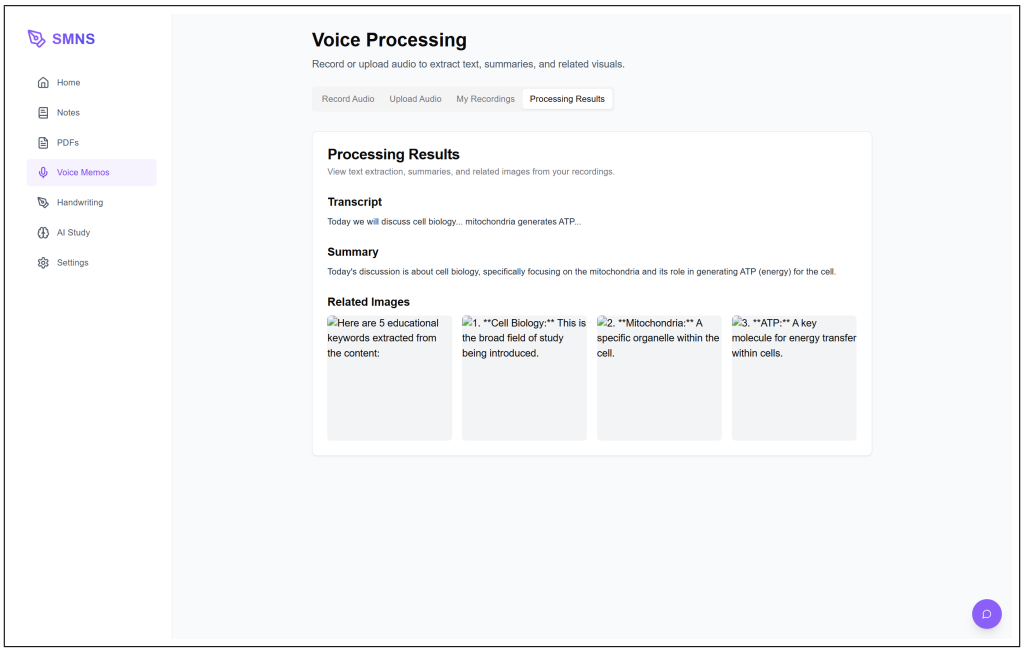



Figure 5.10: Voice Processing

The application incorporates a suite of AI-powered features integrated through a secure Flask backend. These AI modules power the summarization of lengthy lecture notes into concise overviews, automate Q&A generation based on contextual content, and construct quizzes with a mix of multiple-

choice and fill-in-the-blank formats. Carefully designed prompt templates and keyword extraction strategies are employed to ensure that the educational output is not only contextually accurate but also pedagogically effective. These AI-generated assets are then visualized on the dashboard, helping users reinforce learning and organize study material more efficiently.

### 5.1.5 Handwriting Input Module

The handwriting input feature is developed using HTML5 Canvas for the web and React Native SVG for the mobile application. Strokes made by the user are saved as images, which are then processed via OCR or a fine-tuned handwriting recognition model. The extracted text is rendered using a custom font and stored in MongoDB.

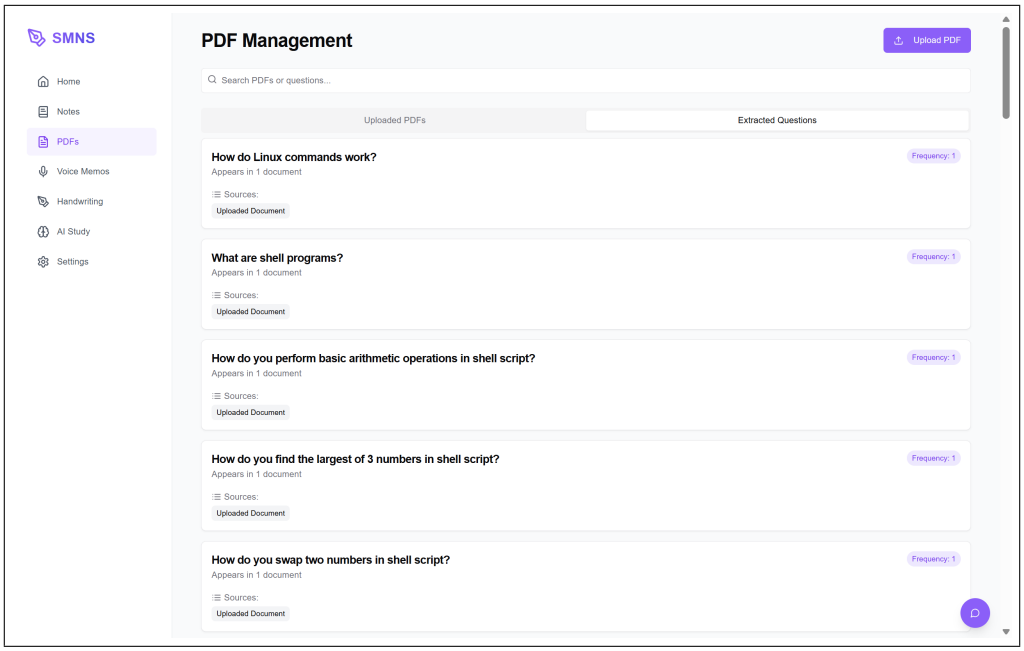



Figure 5.11: Pdf Upload

### 5.1.6 PDF Upload and Question Extraction

The system supports uploading educational content in PDF format, such as class notes or question papers. Upon upload, the backend processes the PDF using tools like PyMuPDF and pdfplumber to extract content. PyMuPDF efficiently handles both text and graphical elements, while pdfplumber excels

in extracting complex layouts, such as tables and multi-column formats.

Once the PDF content is extracted, it is further processed using advanced language models for better understanding. This step helps break the content into smaller parts, identify important topics, and group similar questions together. The system looks at the meaning of each sentence to find questions that may be worded differently but mean the same thing. It also checks for common patterns and repeated topics using smart AI tools. Even if the content is handwritten or a bit messy, the system can still recognize questions correctly. After processing, the final output is shown on the dashboard. Users can see the list of questions, what topics they belong to, how often they appear, and how difficult they might be. This makes it easier to review important content and use features like quiz creation or smart study suggestions. *Refer fig: 5.11 Pdf Upload & 5.12 Pdf Extract.*

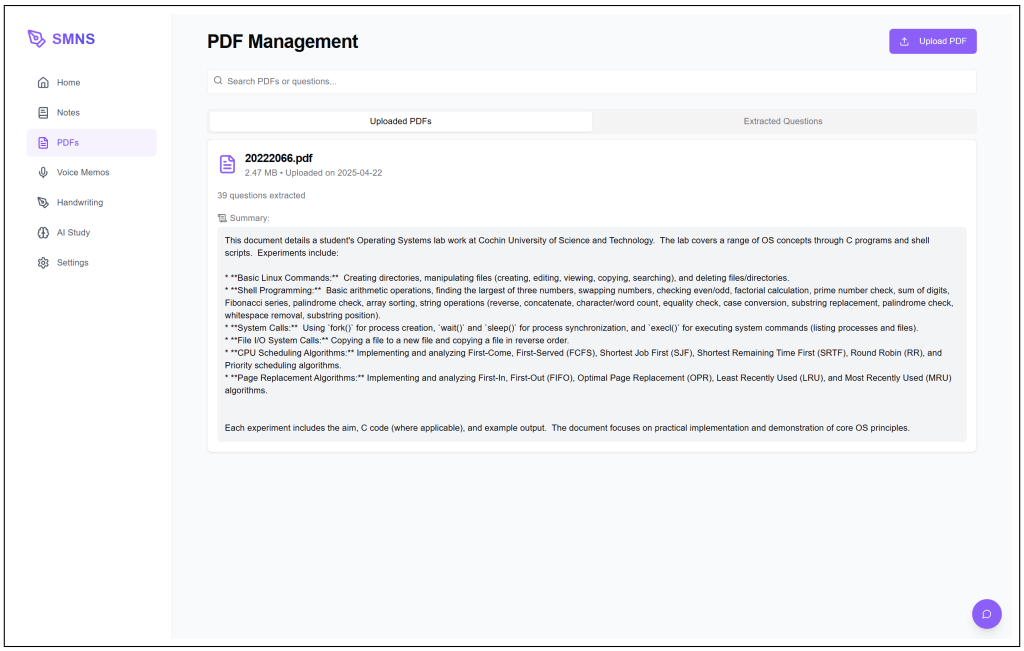



Figure 5.12: Pdf Extract

### 5.1.7 AI-Driven Features

The system leverages structured prompts to generate concise 3–5 sentence summaries, such as transforming a lecture into key points with a prompt like "Summarize the following lecture into 3 key points: ¡note_text¿." The Q&A system processes user questions along with relevant context by sending

them to the Gemini API, which returns accurate and contextually relevant answers. Additionally, the Quiz Generator utilizes TextRank and keyword extraction algorithms to automatically create MCQs and fill-in-the-blank questions. These generated quizzes are stored in the user's dashboard for later access and review.

### 5.1.8 Data Storage and Retrieval

The application utilizes several MongoDB collections, including `notes`, `ai_outputs`, and `audio_transcripts`, to manage the data efficiently. Each document within these collections stores raw user input (such as handwritten notes, audio files, or PDFs), the corresponding AI-generated output (e.g., transcriptions, summaries, or extracted information), and associated metadata (such as user ID, timestamps, and input type). To ensure quick and efficient data retrieval, indexing is performed on key fields like `user_id`, `input_type`, and `timestamp`. This indexing strategy enables the system to filter and search large datasets rapidly, ensuring that user queries and data retrieval processes are handled seamlessly. Furthermore, MongoDB's flexible schema allows the system to scale efficiently as new data types and features are integrated into the application.

# Chapter 6

# Technologies, Stacks and Tools Used

This chapter outlines the various technologies, development stacks, and tools that were utilized in the implementation of the Smart Multi-Note System (SMNS), spanning both the mobile and web platforms.

## 6.1 Development Environment

The development of SMNS relied on a versatile and robust set of tools tailored for both mobile and web platforms. For mobile application development, we used Android Studio, which served as the primary Integrated Development Environment (IDE). It offered extensive support for Android SDKs and libraries, making it ideal for building on top of the Omni-Notes architecture. The web application development was carried out using Visual Studio Code, a lightweight yet powerful code editor.

The core language used for the Android application was Java, while Gradle was employed for managing build automation and dependencies. On the web side, Node.js served as the JavaScript runtime for backend operations, and npm was used for handling JavaScript packages efficiently.

## 6.2 Frontend Technologies

To ensure a seamless, responsive, and platform-consistent user experience, the frontend layer is developed with a focus on modularity, scalability, and usability. The frontend architecture integrates performance-optimized libraries and UI paradigms tailored to each platform mobile and web while maintaining a cohesive visual identity and operational logic. Core attention is given

to delivering fast load times, smooth transitions, intuitive navigation, and accessibility across various screen sizes and devices.

### 6.2.1 Mobile Application Frontend

The frontend of the Android application was developed using native Android components, with layout structures defined in XML. Material Design guidelines were strictly followed to ensure visual consistency, component familiarity, and accessibility. A fragment-based architecture was implemented to promote UI modularity and to support dynamic UI rendering depending on user interactions and device orientation.

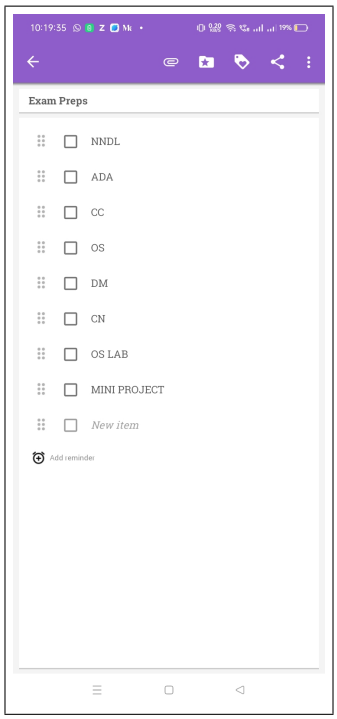



Figure 6.1: Checklisting

RecyclerView was used for rendering scrollable lists of notes and documents, providing memory-efficient rendering of large datasets through view recycling and ViewHolder patterns. The checklisting feature within notes was seamlessly integrated into this component, allowing users to create interactive to-do lists with checkable items directly embedded within their notes. This facilitated task tracking, goal management, and organization within each individual note, supporting a more actionable note-taking experience. To enhance organizational capability, a categorization system was implemented, enabling users to assign notes to specific categories or tags. This allowed

for streamlined retrieval, improved content filtering, and a more intuitive browsing experience within the application. The categorization feature was integrated with the existing data model and UI components, including color-coded labels and dynamic filtering options accessible via the top toolbar and side navigation. *Refer fig: 6.1 Checklisting & 6.2 Categorizing Notes.*

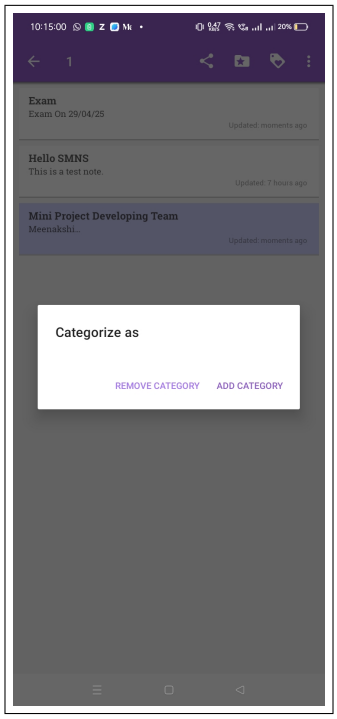



Figure 6.2: Categorizing Notes

To facilitate in-app navigation and improve the user experience, DrawerLayout was incorporated, enabling a side-drawer menu for easy access to different sections such as Notes, Checklist Manager, AI Tools, and User Settings. The navigation drawer dynamically updated to reflect available categories and recent checklist-based notes, providing users with context-aware shortcuts and reducing friction during task switching. The navigation panel was designed with intuitive iconography and adaptive styling to maintain usability across different screen sizes. Additionally, support for backstack-aware navigation ensured a seamless user flow, preserving the context of previous interactions. *Refer fig: 6.3 Navigation.*

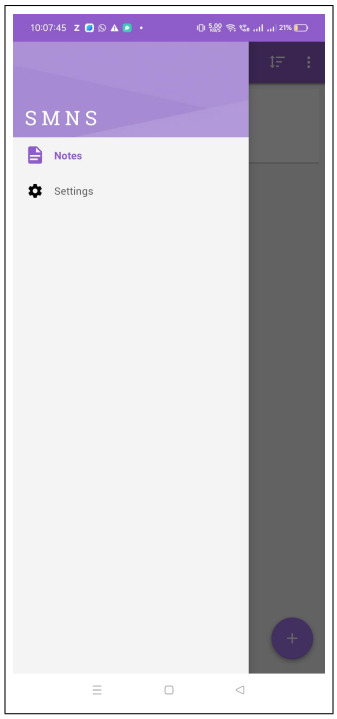



Figure 6.3: Navigation

### 6.2.2 Web Application Frontend

The web interface was developed using React.js, a component-based JavaScript library that supports the creation of interactive and reactive user interfaces. React's virtual DOM architecture ensured optimal rendering performance, especially during frequent data updates like note edits and quiz generations.

HTML5 and CSS3 were used for content structure and style respectively, with Bootstrap incorporated to facilitate a responsive and grid-based layout system. This guaranteed consistent rendering across various browsers and screen resolutions. For navigation, React Router was integrated to enable client-side routing and maintain stateful transitions without full page reloads.

A significant interactive feature implemented in the web frontend was handwriting input, powered by the Canvas API. This allowed users to draw or write notes directly using a stylus or mouse, enhancing natural note-taking behavior, especially for mathematical expressions or diagrams. The resulting strokes were captured as bitmap data, later processed by the backend OCR.

For client-side routing, we utilized React Router, which allowed seamless navigation between pages. One of the standout features was the handwriting input, implemented using the Canvas API, offering a natural and intuitive input experience for users.

## 6.3 Backend and Data Management

To facilitate seamless interaction and ensure consistent data synchronization across both the mobile and web platforms, the system integrates tailored backend architectures optimized for performance, scalability, and maintainability. Each backend setup is aligned with the specific operational requirements of its platform, allowing for efficient data handling and feature expansion.

### 6.3.1 Mobile Application Backend

The Android application's backend is designed with a lightweight yet robust architecture. Persistent data storage is managed using SQLite, a compact, embedded database engine ideal for local storage on mobile devices. This allows offline access to user notes, tags, and metadata without compromising on speed or reliability.

For storing configuration values, theme preferences, and user-specific flags, SharedPreferences is utilized, offering a fast key-value store for primitive data types. File attachments such as images or audio are managed via the local file system, with optional support for periodic backup to the cloud. The backend logic interacts with Android components like Room (where applicable) for database abstraction, ensuring type safety and easier migration support.

### 6.3.2 Web Application Backend

The web application backend is built using Python and follows a modular RESTful API architecture. A minimalist yet powerful framework is used to serve endpoints that handle CRUD operations, user management, and AI interactions. The choice of MongoDB as the primary database offers flexibility through its document-oriented data model, making it easy to store complex objects like nested notes, AI outputs, and transcripts.

Mongoose, a popular ODM (Object Data Modeling) library, simplifies the interaction with MongoDB by providing schema-based modeling, validation, and built-in hooks for lifecycle operations. This ensures clean code and data integrity throughout the application. And for secure user authentication and access control, the system implements JWT (JSON Web Tokens), which allows stateless and scalable user session handling. Tokens carry encrypted claims that verify identity and permissions, making the application both secure and efficient during high-volume requests.

## 6.4 AI Integration Features

The system leverages the Google Gemini API to deliver a suite of advanced AI-powered learning tools, transforming conventional note-taking into a highly interactive and personalized educational experience. At the core of this transformation lies the AI Language Model, which users can fine-tune through AI Preferences, selecting from Precise (focused on accuracy and facts), Balanced (a mix of precision and creativity), or Creative (exploratory and diverse responses) modes, to suit their unique learning styles. *Refer fig: 6.4 Selecting Models.*

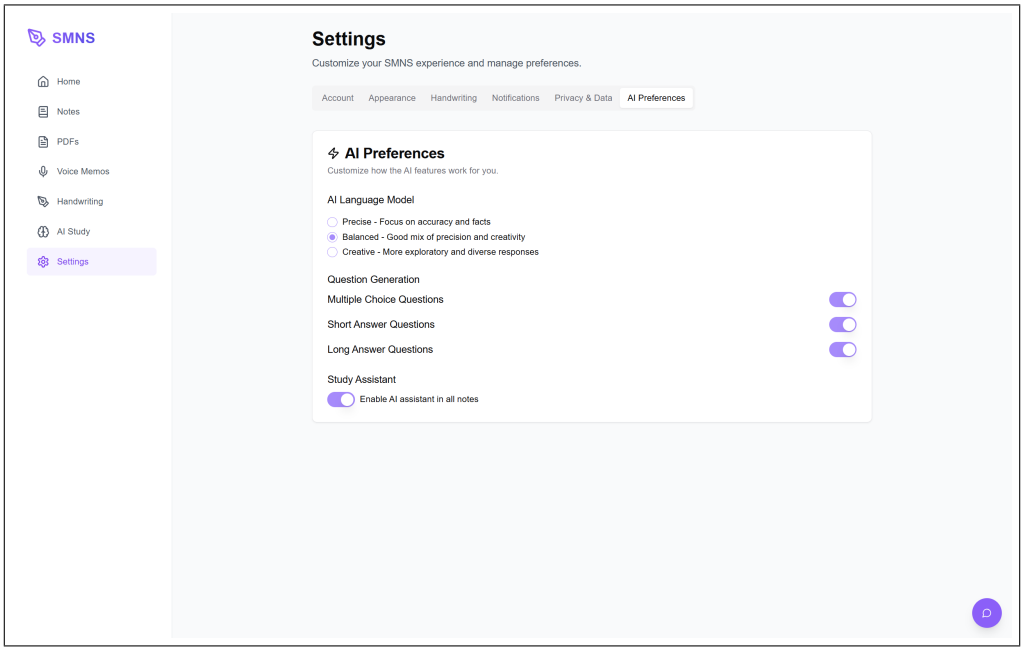



Figure 6.4: Selecting Models

One of the standout features is the intelligent Question Generation capability, which dynamically produces Multiple Choice Questions, Short Answer Questions, and Long Answer Questions from user-generated study materials. This is complemented by a Study Assistant that can be enabled across all notes, providing contextual support, summarizations, and instant clarifications within the learning environment.

Simultaneously, the Whisper API is utilized to transcribe audio inputs such as classroom lectures or spoken memos into searchable and readable digital text, enhancing content accessibility and making revision effortless. Additionally, Optical Character Recognition (OCR) technology is integrated to digitize handwritten notes, thereby bridging the divide between traditional note-taking and modern digital workflows. These combined AI features deliver a truly customized and intelligent educational assistant tailored to individual preferences and academic goals.

## 6.5 Libraries and Dependencies

Both the mobile and web applications for this project utilized several libraries that significantly simplified development and enhanced functionality. These libraries played a crucial role in improving the overall performance and user experience of the applications, reducing boilerplate code, and streamlining communication between components.

### 6.5.1 Mobile Application Libraries

The Android mobile application leveraged several libraries to streamline development and enhance user experience. Butterknife was employed for view binding, reducing the need for manual findViewById calls and minimizing boilerplate code. EventBus facilitated communication between different components in the application, promoting a more modular architecture.

For improving the customization and visual appeal, ColorPickerView was integrated to allow users to select colors for their notes easily. Material Dialog was used to enhance dialog interactions, following Material Design guidelines, providing a smooth and visually appealing way to manage user interactions. Glide, an image loading and caching library, was incorporated to efficiently

handle image display and reduce loading times, ensuring smooth performance and a seamless user experience.

### 6.5.2 Web Application Libraries

The web application made use of a variety of libraries to improve functionality and simplify tasks. Axios, a promise-based HTTP client, was used to manage communication with the backend APIs, ensuring smooth and efficient data retrieval. React Markdown allowed for easy rendering of markdown content into HTML, making it easier to display user-generated notes in a readable format. React PDF was integrated to manage and display PDF documents directly within the application, providing users with the ability to view and interact with their files seamlessly. To handle dates and timestamps, MomentJS was employed, allowing for easy parsing, manipulation, and formatting of date objects, ensuring consistency and accuracy across the application. These libraries not only simplified development but also contributed to an enhanced user interface, making the web app more responsive and feature-rich.

Additionally, React Router was utilized to manage client-side routing, enabling seamless navigation across different sections of the application. Styled-components were adopted for modular and dynamic styling, allowing developers to write CSS directly within JavaScript, thus maintaining a clean and maintainable codebase.

## 6.6 Testing Tools

In the project, Postman was utilized extensively for testing and validating RESTful API endpoints during the backend development phase. Chrome DevTools played a crucial role in real-time debugging of the web application's frontend, enabling efficient identification and resolution of UI and network issues. On the mobile application side, JUnit was used to perform unit testing of core logic to ensure accuracy and prevent regression errors. The React Testing Library was adopted for testing the web interface, focusing on simulating actual user interactions to verify component behavior.

## 6.7 Version Control and Project Management

SMNS employed Git for version control, which facilitated systematic code management and collaborative development among the team. The GitHub repository served as the central platform for maintaining the codebase, tracking issues, and conducting pull request reviews. Task distribution, and progress updates were handled through GitHub Projects, ensuring structured coordination and timely feature delivery throughout the development cycle.

## 6.8 Deployment

The deployment architecture is designed for accessibility and ease of maintenance. The web application is deployed on Heroku, which allowed for seamless CI/CD integration, dynamic resource allocation, and real-time environment monitoring. Backend APIs and the authentication service were containerized and linked to Heroku's runtime, simplifying environment management. MongoDB Atlas was chosen as the cloud-hosted NoSQL database, offering automated backups, real-time performance tracking, and secure connection strings for database access. For the mobile component, the Android APK is compiled using Android Studio's build tools and distributed manually to testers via secure links, enabling direct feedback on mobile functionality and interface performance.

## 6.9 Security Measures

Security considerations were integrated throughout the development to ensure both user data confidentiality and system integrity. Passwords were hashed using secure cryptographic methods before being stored in the database to prevent credential leakage. Input validation mechanisms were implemented across all modules to mitigate risks of injection attacks and ensure robust data handling. HTTPS is enforced for all client-server communication to guarantee encrypted and secure data transmission.

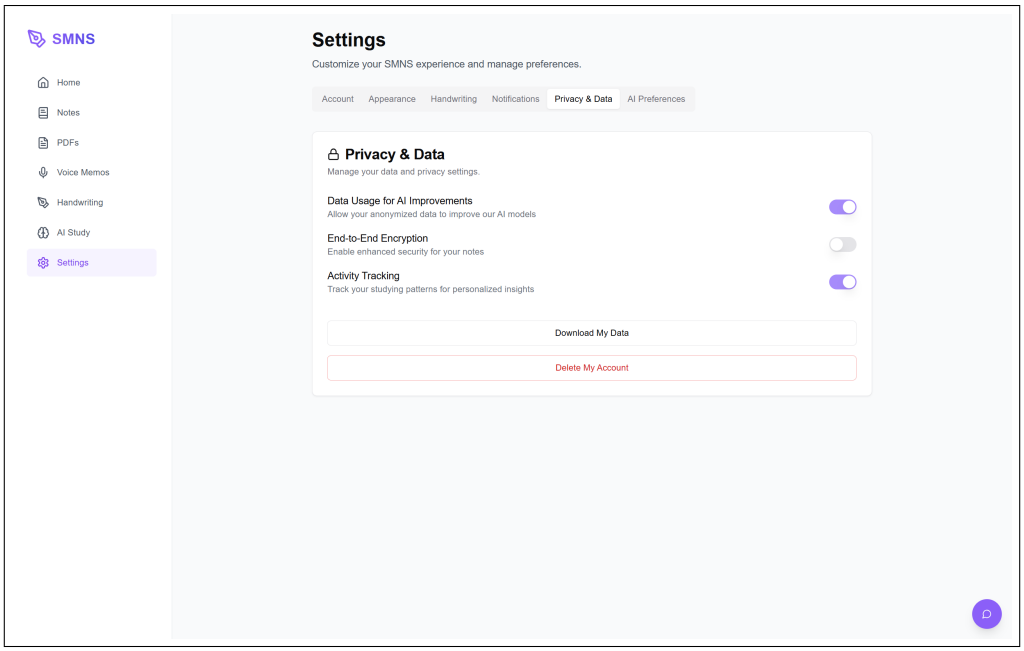



Figure 6.5: Privacy

To further enhance user control and transparency, the application provides a comprehensive settings module. Users can customize their SMNS experience and manage preferences related to appearance, handwriting, notifications, and AI behavior. Privacy and data controls allow users to manage how their data is used, including the option to allow anonymized data for AI improvements. End-to-end encryption is enabled to enhance the security of stored notes. Activity tracking is available for users who wish to gain personalized insights based on study patterns. Additionally, features to download personal data and delete accounts are provided, supporting full user autonomy over their data lifecycle. *Refer fig: 6.5 Privacy.*

# Chapter 7

# Testing

Testing was conducted to validate the functionality, performance, and usability of the SMNS – Smart Multi-Note System. A layered testing methodology was adopted, starting with individual module testing through unit tests, followed by integration testing to ensure smooth interaction between modules, and concluding with performance and usability evaluations under real-world usage scenarios.

## 7.1 Unit Testing

Unit testing was carried out to confirm that the core functionalities of each module operated correctly in isolation. The primary objective was to verify that individual components of the system performed according to their intended design specifications before being integrated into the complete system. For this phase, tools such as Google Gemini and Jest in conjunction with the React Testing Library were employed. Google Gemini enabled extensive API testing and provided robust support for simulating various backend responses and edge cases. On the frontend, Jest and React Testing Library offered a framework to simulate user interactions and verify component behavior in a virtual DOM environment.

The tested modules included components responsible for handling audio transcription, OCR processing of handwritten notes, summarization, question generation. Each module underwent multiple rounds of testing to ensure accuracy, stability, and resilience under a variety of input conditions. Test cases were revisited and re-executed after every update or modification to verify the integrity of the system and to prevent regressions. *Refer to Table 7.1: Unit Testing Results for SMNS Modules.*

**Tested Modules and Results:** The following table summarizes the major modules tested and the corresponding outcomes of their unit tests.

Table 7.1: Unit Testing Results for SMNS Modules

| **Module** | **Test Case Description** | **Result** |
|---|---|---|
| Voice Upload | Accepts audio files and triggers Whisper transcription | Pass |
| Handwriting Input | Checks image capture and OCR integration | Pass |
| PDF Upload | Extracts questions and stores them correctly | Pass |
| Summarization API | Returns 2–4 sentence summary of provided text | Pass |
| Question Generator | Produces MCQs and fill-in-the-blanks | Pass |
| Q&A Assistant | Accepts user questions and responds contextually | Pass |

To ensure smooth transitions between modules, multiple test cases were repeatedly executed after each modification or update.

## 7.2 Integration Testing

Following the successful completion of unit testing for each individual module, integration testing was initiated to assess the overall coherence and functional interoperability. This critical testing phase aimed to verify how effectively different components interacted under real-world academic scenarios. The objective was to ensure that user workflows executed seamlessly and that the system responded reliably to diverse input types and usage patterns.

In the first scenario, a student uploaded a handwritten note, which was then processed using the integrated Optical Character Recognition (OCR) engine. The system accurately extracted text from the image and proceeded to generate a concise summary along with a series of contextually relevant academic questions. This validated the OCR-to-summary pipeline and its applicability for student note digitization.

The second scenario involved uploading an audio recording, such as a recorded lecture or discussion. The system leveraged the Whisper model to transcribe the audio into text, which was subsequently passed to the summarization engine. This confirmed the robustness of the audio-to-text conversion flow and its integration with downstream processing modules.

In another use case, a PDF document containing multiple academic questions was uploaded. The system effectively parsed the document, identified recurring question patterns, and presented the findings within the user dashboard. This scenario confirmed the ability of the SMNS to handle document analysis and visual data representation for academic materials.

Lastly, a user input a custom academic query. The LLM-powered Q&A assistant accurately interpreted the question and returned a precise, context-aware answer. This validated the natural language understanding and response generation capabilities of the system, affirming its relevance as a learning assistant.

All integration tests yielded stable and consistent outcomes. The workflows were thoroughly validated on both Android and web platforms, with successful replication by multiple testers, thereby ensuring the reliability, cross-platform compatibility, and user-centered design integrity of the Smart Multi-Note System.

## 7.3 Performance Testing

To ensure the Program could maintain stability and efficiency under realistic usage conditions, a series of performance tests including stress testing and load testing were conducted. These tests were designed to evaluate

how well the system handled multiple user actions simultaneously, processed large media inputs, and maintained consistent performance under pressure. Key performance indicators such as response time, processing speed, system throughput, and resource utilization were closely monitored during these assessments. *Refer to Table 7.2: Performance Testing Results.*

Table 7.2: Performance Testing Results

| **Test Scenario** | **Result** |
|---|---|
| Simultaneous login of 50+ users | No slowdowns or crashes |
| Uploading large 10-minute audio file | Transcription under 15 seconds |
| Summarizing 1000-word note | Completed in less than 5 seconds |
| Average API response time | 350–700 ms (within acceptable range) |

These results validate the system's readiness for real-world deployment, offering seamless performance under moderate to high workloads.

## 7.4 Usability Testing

To assess the ease of use and overall user experience, usability testing was performed with a diverse group of 10 students from different departments and academic backgrounds. Feedback Overview: Approximately 90% of users found the app intuitive and user-friendly. Around 75% indicated that the app's summaries improved their study preparation speed. Furthermore, 70% of testers regularly engaged with the Q&A assistant for clarifications. Notably, 100% of participants expressed interest in using the application if officially released.

## 7.5 Bug Fixes and Improvements

During the testing process, several bugs and performance bottlenecks were identified. Each issue was carefully resolved to ensure smooth functioning

and better user experience.The table below highlights major bugs and the improvements made during testing iterations. *Refer to Table 7.3 Bug Fixes and Enhancements.*

Table 7.3: Bug Fixes and Enhancements

| **Issue Identified** | **Action Taken** |
|---|---|
| Repeated API calls for single button tap | Debounced actions using time-out control |
| Long audio crashing on mobile | Added size compression and asynchronous handling |
| Gemini slow on large input | Optimized prompt using input chunking |

These improvements significantly boosted system efficiency and user satisfaction across multiple test cycles.

# Chapter 8

# Future Scope

While the current version of the app establishes a solid foundation, several strategic enhancements are planned to expand its impact in academia. Key among these is real-time collaboration, enabling study groups or project teams to annotate and edit shared notes simultaneously, promoting interactive peer learning. Multilingual support, powered by Gemini's language models, will make the app more inclusive by accommodating regional and global languages. To ensure continuity in learning, cloud sync and offline access will allow users to work seamlessly across connectivity states. A learning analytics dashboard will provide insights into usage patterns such as frequently revised topics and AI interaction trends empowering users to adopt more effective study strategies. Integration with Learning Management Systems (LMS) like Moodle and Google Classroom will automate course content import, while a teacher mode will allow educators to generate summaries and question banks aligned with their syllabus.

Lastly, robust privacy controls including encryption and role-based access will safeguard user data, ensuring compliance with modern data protection standards.

# Chapter 9

# Conclusion

The Smart Multi-Note System (SMNS) stands as a testament to how artificial intelligence can redefine traditional learning tools by offering an intelligent, adaptive, and interactive approach to note-taking and study management. The system amalgamates multiple technologies handwriting recognition, audio transcription, and PDF analysis into a cohesive platform powered by AI capabilities such as summarization, Q&A generation, mind mapping, and quiz creation. These features not only enhance the user experience but also significantly streamline study workflows.

A key focus throughout development was on building a modular and scalable architecture, ensuring ease of expansion and maintenance. The frontend was crafted using React JS and TypeScript, prioritizing clarity and responsiveness in the user interface. On the backend, robust AI models were employed, including Whisper for audio transcription and the Gemini API for natural language processing tasks. Comprehensive testing including unit, integration, performance, and usability evaluations was conducted to validate the system's reliability and effectiveness. These efforts ensured the platform meets the standards of both functionality and user satisfaction.

In summary, SMNS empowers students by offering a smarter, faster, and more personalized way to engage with their learning materials. It paves the way for a more curious, self-directed, and efficient approach to education in the digital era.

# References

1. R. Vishnu Vardhan Reddy and Dr. G. Rajeswari , *Automated Document Processing: Combining OCR and Generative AI for Efficient Text Extraction and Summarization*, IJRTI , March 2025 .

2. Radford et al., *Whisper: Robust Speech Recognition via Whisper*, OpenAI, 2022. https://openai.com/research/whisper

3. O. Zawacki-Richter, V. I. Marín, M. Bond, and F. Gouverneur, "Systematic review of research on artificial intelligence applications in higher education," *International Journal of Educational Technology in Higher Education*, vol. 16, no. 1, pp. 1–27, 2019.

4. R. Nallapati, B. Zhou, C. Gulcehre, and B. Xiang, "Abstractive Text Summarization Using Sequence-to-Sequence RNNs and Beyond," in *Proceedings of The 20th SIGNLL Conference on Computational Natural Language Learning (CoNLL 2016)*, 2016.

5. Google DeepMind, *Gemini API – Multimodal Language Model Platform*, 2024. https://ai.google.dev